

Universidad Nacional de Asunción
Facultad Politécnica
Ingeniería en Informática



Trabajo Final de Grado

**Detección de anomalías y clasificación de
ataques en tráfico HTTP: comparación entre
enfoques one-class, multiclase y multitiqueta**

Autores:

Andrés Román
Juan González

Tutor:

Dr. Cristian Cappelletti

SAN LORENZO - PARAGUAY
ENERO - 2026

Resumen

Se presenta un flujo reproducible de aprendizaje automático orientado a la detección de anomalías y a la clasificación de ataques en tráfico HTTP. La comparación experimental se organiza sobre tres formulaciones complementarias del problema: detección *one-class*, clasificación multiclase y clasificación multietiqueta. El alcance es experimental y comparativo. No se implementa ni se despliega un *Web Application Firewall* completo; en su lugar, se examina la factibilidad teórica de utilizar los modelos evaluados en un contexto tipo WAF a partir de métricas de efectividad y de costo operativo.

La representación de entrada se construye sobre una base común de peticiones HTTP, definida mediante variables interpretables extraídas del método, la URI, el cuerpo y, cuando está disponible, un campo derivado de encabezados. Esa base se materializa en un conjunto fijo de veinticinco características que describen longitud, estructura, composición de caracteres, codificación, señales léxicas y atributos básicos del protocolo.

La comparación experimental se distribuye sobre tres fuentes de datos con distinta granularidad de etiquetado: CSIC 2010, útil principalmente para detección binaria; CSIC/TorpEda 2012, apto para detección binaria y clasificación multiclase; y SR-BH 2020, un dataset de tráfico real de honeypot con anotación CAPEC multietiqueta. Para preservar comparabilidad entre escenarios, la implementación normaliza los campos de entrada y deriva las variables `label_binary`, `label_multiclass` y `label_multilabel` según la información disponible en cada dataset.

Los modelos seleccionados responden a criterios de escalabilidad, trazabilidad e interpretabilidad. Para detección *one-class* se utiliza un Pipeline compuesto por `StandardScaler`, `Nystroem` y `SGDOneClassSVM`, entrenado únicamente con tráfico normal. Para las tareas supervisadas se emplea regresión logística lineal, mientras que para clasificación multietiqueta se adopta la descomposición *One-vs-Rest*. El protocolo experimental aísla el conjunto de prueba externo, restringe el ajuste de hiperparámetros al bloque de entrenamiento y contempla tanto métricas de clasificación como métricas operativas, entre ellas latencia, *throughput*, consumo de CPU y memoria, tamaño de modelo y tiempo de entrenamiento.

La metodología incorpora además una comparación directa entre corridas con el conjunto completo de variables y corridas en las que se eliminan únicamente las variables basadas en tokens sospechosos. La comparación permite distinguir con mayor precisión cuánto del rendimiento observado proviene de señales estructurales del tráfico HTTP y cuánto depende de indicadores léxicos más cercanos a reglas o firmas.

Abstract

A reproducible machine learning pipeline for anomaly detection and attack classification in HTTP traffic is presented. The experimental comparison covers three complementary problem formulations: *one-class* detection, multiclass classification, and multilabel classification. The scope is experimental and comparative. A full *Web Application Firewall* is neither implemented nor deployed; instead, the theoretical feasibility of using the evaluated models in a WAF-like setting is examined through effectiveness and operational-cost metrics.

The input representation is built from a common description of HTTP requests based on interpretable variables extracted from method, URI, body, and, when available, a header-derived field. This design is materialized through a fixed set of twenty-five features describing length, structure, character composition, encoding, lexical cues, and basic protocol attributes.

The experimental comparison is organized around three data sources with different labeling granularity: CSIC 2010, mainly useful for binary detection; CSIC/TorpEda 2012, suitable for binary detection and multiclass classification; and SR-BH 2020, a real honeypot dataset with CAPEC multilabel annotations. To preserve comparability across scenarios, the implementation normalizes the input fields and derives the variables `label_binary`, `label_multiclass`, and `label_multilabel` according to the information available in each dataset.

The selected models follow the criteria of scalability, traceability, and interpretability. For *one-class* detection, the implementation uses a `Pipeline` composed of `StandardScaler`, `Nystroem`, and `SGDOneClassSVM`, trained only with normal traffic. For supervised tasks, linear logistic regression is used, while multilabel classification is handled through the *One-vs-Rest* decomposition. The experimental protocol isolates an outer test set, restricts hyperparameter tuning to the training block, and considers both classification metrics and operational metrics such as latency, throughput, CPU and memory usage, model size, and training time.

The methodology also includes a direct comparison between runs using the complete feature set and runs in which only suspicious-token-based variables are removed. This makes it possible to discuss more clearly how much of the observed performance comes from structural HTTP signals and how much depends on lexical indicators that are closer to signatures or rules.

Agradecimientos

Índice general

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Planteamiento del problema	1
1.3. Objetivos	1
1.3.1. Objetivo general	1
1.3.2. Objetivos específicos	2
1.4. Contribuciones	3
2. Marco teórico	4
2.1. HTTP y superficie de ataque	4
2.2. WAF: enfoque por firmas vs anomalías	4
3. Base matemática de los modelos	5
3.1. Notación y definición exacta de variables	5
3.1.1. Dataset, features y objetivos	5
3.1.2. Tabla de símbolos (resumen)	6
3.1.3. Correspondencia con la implementación	6
3.2. Detector one-class escalable	6
3.2.1. Intuición	6
3.2.2. Implementación adoptada	6
3.2.3. Justificación metodológica	7
3.2.4. Implicancias para el diseño experimental	7
3.3. Regresión logística	7
3.3.1. Caso binario	7
3.3.2. Función de pérdida (log-loss) con regularización	8
3.3.3. Figura de la sigmoide	8
3.3.4. Multiclase (softmax)	8
3.3.5. Decisiones del proyecto	9
3.4. Clasificación multietiqueta con One-vs-Rest (OvR)	9
3.5. Métricas y base matemática	9
3.5.1. Binario	9
3.5.2. Multiclase: accuracy, promedios micro/macro/weighted y MCC	10
3.5.3. Multietiqueta: Hamming Loss, Jaccard y coincidencia exacta	11
3.5.4. Métricas temporales y operativas	12
3.5.5. Diccionario resumido de métricas retenidas	13

4. Datasets y criterios de selección	15
4.1. CAPEC como esquema semántico de ataques	15
4.2. SR-BH 2020: tráfico real de honeypot con etiquetas CAPEC	15
4.2.1. SR-BH 2020 como referencia de tráfico real	15
4.3. Datasets alternativos y unificación a un esquema común	15
4.3.1. Estrategia de unificación (implementación)	16
4.3.2. comparativa de la tríada de datasets	16
4.3.3. Reducción multietiqueta a multiclase	16
5. Representación de las peticiones HTTP	17
5.1. Principio general	17
5.2. Lista de features implementada	17
5.2.1. Cómo se construye cada feature	18
5.2.2. Lectura metodológica de estas variables	20
5.3. Criterios de diseño de la representación	20
6. Metodología y diseño experimental	21
6.1. Tareas definidas	21
6.2. Matriz experimental (3 modelos $\times$ 3 datasets)	21
6.3. Política general de particionado y reproducibilidad	22
6.4. Particionado por tarea	22
6.4.1. Supervisado (multiclase y multietiqueta)	22
6.4.2. Detector one-class sin contaminación	23
6.5. Modelos seleccionados	23
6.6. Selección de hiperparámetros (Grid/Random Search) y validación	24
6.6.1. Tuning en supervisado (multiclase y multietiqueta)	24
6.6.2. Tuning en el detector one-class	24
6.7. Configuración experimental efectivamente utilizada	24
6.7.1. Construcción y preparación de los datasets	25
6.7.2. Detector one-class en CSIC 2010, CSIC/TorpEda 2012 y SR-BH 2020	25
6.7.3. Regresión logística multiclase en CSIC 2010, CSIC/TorpEda 2012 y SR-BH 2020	25
6.7.4. OvR lineal para multietiqueta y modos reducidos por dataset	26
6.7.5. Lectura metodológica conjunta	26
6.8. Métricas (efectividad y eficiencia)	27
6.9. Criterios de comparación con la literatura	28
6.10. Importancia de características (FI)	28
6.11. Amenazas a la validez	29
7. Implementación (pipeline)	30
7.1. Arquitectura del pipeline	30
7.2. Artefactos generados	30
7.3. Derivación de <code>label_multiclass</code> a partir de CAPEC (SR-BH)	30
8. Resultados	32
8.1. Criterio de presentación	32
8.2. Lectura metodológica de las corridas	32
8.3. Detección one-class	33
8.3.1. Métricas de efectividad	33

8.3.2. Métricas operativas	33
8.4. Clasificación multiclase	34
8.4.1. Métricas de efectividad	34
8.4.2. Métricas operativas	34
8.5. Clasificación multietiqueta	35
8.5.1. Métricas globales	35
8.6. Síntesis transversal	37
8.7. Limitaciones de las corridas	37
9. Protocolo de evaluación de viabilidad en tiempo real en un entorno tipo WAF	39
9.1. Requisitos operativos	39
9.2. Consolidado operativo por dataset y modelo	39
9.3. Lectura de factibilidad	40
10. Conclusiones y trabajo futuro	41
10.1. Conclusiones	41
10.2. Trabajos futuros	41
A. Comandos reproducibles	42

Índice de figuras

3.1.	Función sigmoide usada por regresión logística [13].	8
3.2.	Esquema One-vs-Rest para multietiqueta: un clasificador binario por etiqueta [12, 11].	9
7.1.	Pipeline propuesto: unificación de datasets, extracción de features, entrenamiento escalable, tuning con presupuesto fijo y evaluación.	30

Capítulo 1

Introducción

1.1. Contexto y motivación

Un *Web Application Firewall* (WAF) inspecciona conversaciones HTTP y aplica políticas para mitigar ataques comunes como SQL Injection y XSS [1]. En la práctica, los WAF basados únicamente en firmas/reglas requieren mantenimiento continuo y tienden a degradarse frente a variaciones de *payload* o ataques novedosos. ModSecurity es un ejemplo de motor WAF ampliamente usado para monitoreo y reglas [2].

Por otro lado, los enfoques basados en anomalías aprenden una representación del tráfico “normal” y detectan desviaciones, lo cual es útil cuando la cobertura de ataques es incompleta o cambiante. En el dominio web, trabajos clásicos como Kruegel y Vigna muestran que atributos estadísticos y de estructura de requests permiten puntuar anomalías [10].

El planteamiento adoptado toma como base un enfoque por anomalías y lo extiende con un dataset de tráfico HTTP real con etiquetas de ataque estructuradas (CAPEC), habilitando comparaciones entre *one-class* y enfoques supervisados multiclase/multietiqueta.

1.2. Planteamiento del problema

En entornos reales existe: (i) alta variabilidad del tráfico legítimo, (ii) fuerte desbalance entre normales y ataques, y (iii) ataques emergentes que evolucionan. Un sistema útil debe:

- detectar anomalías cuando hay pocos datos rotulados (o cambian los ataques), y
- cuando hay etiquetas, clasificar el tipo/patrón para respuesta e investigación.

1.3. Objetivos

1.3.1. Objetivo general

El objetivo general es diseñar, implementar y evaluar un *pipeline* reproducible de aprendizaje automático para (i) detección de anomalías/ataques en tráfico HTTP y (ii) clasificación de ataques por tipo (multiclase y, cuando aplique, multietiqueta), comparando un dataset de tráfico **real** recolectado en honeypot (SR-BH 2020, Harvard Dataverse) con datasets **sintéticos/benchmark** (CSIC 2010 y CSIC/TorpEda 2012) para estudiar *domain*

shift. Además, se analizará la **viabilidad de implementación en tiempo real** del pipeline (extracción de features + inferencia) en un entorno tipo WAF, midiendo eficiencia (latencia P50/P95/P99, throughput, CPU/RAM, tamaño del modelo, tiempo de entrenamiento) y funcionamiento (TPR/FPR y trade-offs FP/FN), y se compararán los resultados con trabajos relacionados usando un conjunto común de métricas [9, 24, 25, 26, 27, 28, 29].

En términos experimentales, la comparación se construye sobre una misma representación de peticiones HTTP aplicada a tres datasets y a tres formulaciones del problema. El contraste no se limita al rendimiento predictivo: incorpora también costo computacional, estabilidad y trazabilidad de las decisiones, de modo que la discusión final distinga con claridad entre el aporte de cada modelo y las exigencias propias de cada escenario de datos.

1.3.2. Objetivos específicos

- Implementar un pipeline de extracción de **25 características** desde método, URI, cuerpo HTTP y, cuando esté disponible, un campo derivado de *headers* (`req_content_length`), alineado con literatura de detección en requests.
- Unificar y evaluar tres fuentes de datos: SR-BH 2020 (tráfico real de honeypot con etiquetas CAPEC) [8, 9], y dos datasets sintéticos/benchmark (CSIC 2010 y CSIC/TorpEda 2012) [21, 23, 22].
- Entrenar y comparar **tres configuraciones/modelos** sobre los **tres datasets, adaptando la tarea a la disponibilidad de etiquetas** de cada dataset: (i) un detector one-class escalable para detección binaria (normal/anómalo) entrenado solo con normales, implementado como `StandardScaler + Nystroem + SGDOneClassSVM` [18, 19]; (ii) regresión logística supervisada lineal como baseline eficiente [13, 14] (multiclase cuando el dataset provea tipo de ataque o etiqueta primaria; en CSIC 2010 se restringe a binario); (iii) One-vs-Rest (OvR) como baseline multi-salida [12, 11] (multietiqueta CAPEC en SR-BH; en TorpEda se usa en modo multiclase o binario cuando corresponda; en CSIC 2010 se restringe a binario).
- Definir una metodología experimental **sin contaminación entre entrenamiento, validación y prueba**: en supervisado, usar particionado externo **80/20** del total del dataset; y en el detector one-class, entrenar **solo con el 80 % del tráfico normal** y evaluar sobre un conjunto mixto compuesto por el **20 % de normales en holdout** y el **total de anomalías/ataques disponibles**, realizando la selección de hiperparámetros **solo sobre el conjunto habilitado para entrenamiento** y con presupuesto computacional fijo.
- Reportar **métricas de efectividad** alineadas con trabajos relacionados: Accuracy, Precision/Recall/F1 (micro/macro/weighted), TPR/FPR, Specificity, ROC-AUC y PR-AUC [28], además de MCC como métrica robusta al desbalance [29]. En multietiqueta: Hamming Loss, Jaccard Similarity y Exact Match Ratio [9, 12].
- Medir **métricas de eficiencia/tiempo real**: latencia P50/P95/P99 del pipeline (feature extraction + inferencia), throughput (req/s), CPU/RAM, tamaño del modelo y tiempo de entrenamiento; y estabilidad bajo carga [24].
- Analizar el aporte de las *features* mediante una estrategia eficiente: coeficientes estandarizados en los modelos lineales, ablaciones por grupos y comparación **con y sin tokens sospechosos**, priorizando interpretabilidad con costo computacional controlado.

- Comparar resultados con los reportados en literatura sobre CSIC/TorpEda/SR-BH, alineando tareas y métricas, y documentando amenazas a validez cuando difieran *splits*, preprocesamiento o definición de etiquetas [25, 26, 27].

1.4. Contribuciones

- Pipeline reproducible de preprocesamiento y extracción de características HTTP (25 features) a partir de método, URI, body y un campo derivado de headers cuando esté disponible.
- Definición e implementación de etiquetas `label_binary`, `label_multiclass` y `label_multilabel` para tareas comparables entre datasets.
- Política explícita para derivar `label_multiclass` desde la anotación CAPEC multietiqueta de SR-BH mediante una regla determinista basada en severidad, conservando columnas auxiliares de auditoría para revisar la etiqueta primaria elegida.
- Adopción de un detector *one-class* escalable basado en `SGDOneClassSVM` con aproximación de kernel, manteniendo el criterio de entrenar exclusivamente con tráfico normal.
- Baselines supervisados lineales implementados en `scikit-learn`: regresión logística para tareas multiclase y *One-vs-Rest* para salidas multietiqueta, con configuraciones compatibles con el tamaño de los datasets y con análisis posterior por coeficientes.
- Selección opcional de hiperparámetros mediante búsquedas acotadas dentro del bloque de entrenamiento, usando el esquema que corresponda a cada script y guardando los resultados de búsqueda para trazabilidad [16, 20].
- Importancia de características basada en coeficientes estandarizados y ablaciones por grupos, manteniendo el análisis interpretativo dentro del presupuesto computacional del estudio.
- Documento técnico con análisis crítico de decisiones de modelado, compromisos entre costo y rendimiento, y amenazas a la validez.

Capítulo 2

Marco teórico

2.1. HTTP y superficie de ataque

Una petición HTTP contiene método, URI (path y query), cabeceras y, en algunos casos, cuerpo. Los ataques web tienden a manifestarse como patrones de estructura y codificación (p.ej., percent-encoding), longitudes atípicas y presencia de tokens específicos en URI/body. Los WAF típicamente inspeccionan esos campos para aplicar políticas [1, 2].

2.2. WAF: enfoque por firmas vs anomalías

- **Firmas/reglas:** alta precisión para ataques conocidos, pero requiere mantenimiento continuo y puede fallar ante variantes.
- **Anomalías:** aprende el soporte del tráfico normal y detecta desviaciones; puede detectar lo novedoso, pero puede elevar falsos positivos si el normal es muy diverso.

En el dominio web, se han propuesto enfoques de detección de anomalías basados en atributos del request (longitudes, distribución de caracteres, etc.) [10].

Capítulo 3

Base matemática de los modelos

Este capítulo presenta la formulación matemática de los modelos utilizados y fija una notación única para que las variables de las ecuaciones correspondan de manera directa con la implementación.

3.1. Notación y definición exacta de variables

3.1.1. Dataset, features y objetivos

Sea un conjunto de n peticiones HTTP (requests). Cada request se transforma a un vector numérico de d características (features) extraídas de método, URI y cuerpo. Definimos:

- $x_i \in \mathbb{R}^d$: vector de features de la request i (una fila del dataset).
- $X \in \mathbb{R}^{n \times d}$: matriz de diseño; su fila i es $x_i^\top$ (todas las requests juntas).

La variable objetivo depende de la tarea:

- **Binaria:** $y_i \in \{0, 1\}$, donde $y_i = 0$ indica tráfico normal y $y_i = 1$ indica ataque/anómalo. El vector completo es $y \in \{0, 1\}^n$.
- **Multiclase:** $y_i \in \{1, \dots, K\}$, donde K es el número de clases (por ejemplo, NORMAL + clases CAPEC o un agrupamiento). El vector completo es $y \in \{1, \dots, K\}^n$.
- **Multietiqueta:** $y_i \in \{0, 1\}^L$, donde L es el número de etiquetas posibles. El componente $y_{i\ell} = 1$ indica que la request i pertenece a la etiqueta ℓ . El conjunto completo se representa como $Y \in \{0, 1\}^{n \times L}$.

Nota (disponibilidad de etiquetas): no todos los datasets soportan todas las tareas. CSIC 2010 ofrece solo binario (normal/anómalo); TorpEda 2012 ofrece multiclase (tipo de ataque) y su versión binaria por colapso; SR-BH 2020 ofrece binario, multiclase (vía etiqueta primaria) y multietiqueta CAPEC. En consecuencia, al ejecutar los modelos sobre los tres datasets, las tareas multiclase/multietiqueta se evalúan en su versión reducida cuando el dataset no provee la granularidad requerida.

Además, usamos:

- $\hat{y}$ o $\hat{Y}$: predicciones del modelo (mismo tipo/forma que y o Y).
- $\mathbb{I}[\cdot]$: función indicadora; vale 1 si la condición es verdadera y 0 si es falsa.

3.1.2. Tabla de símbolos (resumen)

Símbolo	Significado exacto
n	cantidad total de requests del dataset
d	cantidad de features numéricas por request
K	cantidad de clases en multiclase
L	cantidad de etiquetas en multietiqueta
$x_i \in \mathbb{R}^d$	features de la request i
$X \in \mathbb{R}^{n \times d}$	matriz con todas las requests (filas) y features (columnas)
y_i	etiqueta (binaria o multiclase) de la request i
$y \in \{0, 1\}^n$	vector de etiquetas binarias de todas las requests
$y \in \{1, \dots, K\}^n$	vector de etiquetas multiclase de todas las requests
$y_i \in \{0, 1\}^L$	vector de etiquetas (multietiqueta) para la request i
$Y \in \{0, 1\}^{n \times L}$	matriz multietiqueta para todo el dataset
$\hat{y}, \hat{Y}$	predicciones del modelo
$\mathbb{I}[\cdot]$	función indicadora

3.1.3. Correspondencia con la implementación

En la implementación, una fila x_i corresponde a un conjunto fijo de 25 features (longitudes, estructura, codificación, tokens y codificación one-hot del método). En SR-BH 2020, el dataset publicado incluye 24 features; en la versión adoptada para esta evaluación se recalcula el esquema completo y se agrega una feature derivada de headers (`req_content_length`) para unificar el conjunto en 25. Las variables objetivo corresponden a: `label_binary` (binario), `label_multiclass` (multiclase) y `label_multilabel` (multietiqueta). En SR-BH, `label_multiclass` se deriva desde `label_multilabel` mediante una política explícita basada en severidad. Para mantener trazabilidad, el dataset procesado puede acompañar esa variable con `label_multiclass_first` (auditoría) y `label_primary_severity` (puntaje asociado a la etiqueta primaria).

3.2. Detector one-class escalable

3.2.1. Intuición

El enfoque *one-class* modela el soporte del tráfico normal y marca como anómalas las requests que quedan fuera de esa región. Conceptualmente, esto conserva la motivación original de los WAF basados en anomalías: aprender qué aspecto tiene el tráfico legítimo HTTP y detectar desvíos aun cuando no exista un catálogo completo de ataques.

3.2.2. Implementación adoptada

La implementación seleccionada para todo el protocolo experimental es:

`StandardScaler` $\rightarrow$ `Nystroem` $\rightarrow$ `SGDOneClassSVM`.

Sea $x \in \mathbb{R}^d$ el vector de características HTTP extraídas de una request. Primero se aplica un escalado lineal y luego una transformación explícita $z(x) \in \mathbb{R}^m$ mediante aproximación de kernel. Sobre esa representación, el detector aprende un hiperplano que separa el tráfico

normal del origen en el espacio transformado. La función de decisión puede expresarse como:

$$f(x) = \text{sign}(w^\top z(x) - \rho).$$

Esta formulación preserva el supuesto central del problema —entrenar solo con tráfico normal— y, al mismo tiempo, mantiene el costo de entrenamiento e inferencia en un rango compatible con datasets del orden de 10^6 requests [18, 19].

3.2.3. Justificación metodológica

La elección de esta implementación responde a tres criterios:

1. **Escalabilidad:** el backend basado en SGD permite entrenar con grandes volúmenes sin que el costo crezca de manera impracticable.
2. **Consistencia con el problema:** el detector sigue siendo *one-class*, por lo que se ajusta únicamente con tráfico normal y se evalúa luego sobre tráfico mixto.
3. **Compatibilidad con tiempo real:** el pipeline final se integra naturalmente a la medición de latencia, *throughput*, CPU/RAM y tamaño del modelo exigida por el protocolo experimental.

3.2.4. Implicancias para el diseño experimental

A partir de esta definición, el capítulo experimental adopta una única variante *one-class* para todos los datasets y corridas comparativas. Con ello se mantiene una traza experimental única, se evita duplicar familias conceptualmente solapadas y se conserva el foco en un protocolo ejecutable para el tamaño de datos objetivo.

3.3. Regresión logística

3.3.1. Caso binario

La regresión logística modela la probabilidad de ataque (clase 1) como:

$$P(y = 1 \mid x) = \sigma(z), \quad z = w^\top x + b, \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Definición exacta:

- $x \in \mathbb{R}^d$: vector de features de una request.
- $w \in \mathbb{R}^d$: vector de pesos (importancia de cada feature).
- $b \in \mathbb{R}$: sesgo (intercept).
- $z \in \mathbb{R}$: score lineal (antes de la sigmoide).
- $P(y = 1 \mid x)$: probabilidad estimada de que la request sea ataque.

3.3.2. Función de pérdida (log-loss) con regularización

Para un dataset binario $\{(x_i, y_i)\}_{i=1}^n$ con $y_i \in \{0, 1\}$ y $p_i = P(y = 1 \mid x_i)$, la pérdida promedio es:

$$\mathcal{L}(w, b) = -\frac{1}{n} \sum_{i=1}^n \left(y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right) + \frac{\lambda}{2} \|w\|^2.$$

Definición exacta:

- $\mathcal{L}(w, b)$: función objetivo a minimizar.
- $p_i = \sigma(w^\top x_i + b)$.
- $\lambda \geq 0$: fuerza de regularización $L2$ (penaliza pesos grandes).

3.3.3. Figura de la sigmoide

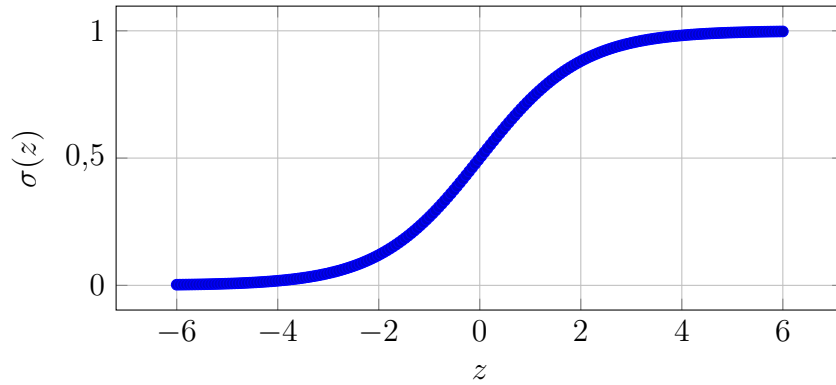


Figura 3.1: Función sigmoide usada por regresión logística [13].

3.3.4. Multiclase (softmax)

Para K clases, el modelo asigna un score por clase:

$$s_k(x) = w_k^\top x + b_k, \quad k = 1, \dots, K,$$

y convierte esos scores en probabilidades con softmax:

$$P(y = k \mid x) = \frac{\exp(s_k(x))}{\sum_{j=1}^K \exp(s_j(x))}.$$

Definición exacta:

- $w_k \in \mathbb{R}^d$: vector de pesos de la clase k .
- $b_k \in \mathbb{R}$: sesgo de la clase k .
- $s_k(x) \in \mathbb{R}$: score lineal para la clase k .
- $P(y = k \mid x)$: probabilidad predicha de la clase k .

La pérdida típica (entropía cruzada multiclase) sobre $\{(x_i, y_i)\}_{i=1}^n$ es:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \log P(y = y_i \mid x_i) + \frac{\lambda}{2} \sum_{k=1}^K \|w_k\|^2.$$

3.3.5. Decisiones del proyecto

- **Baseline supervisado simple e interpretable:** establece una línea base clara antes de modelos más complejos [13].
- **class_weight="balanced" (multiclase):** re-pondera clases inversamente a su frecuencia en el entrenamiento [14].
- **Escalado:** mejora estabilidad numérica y convergencia del optimizador [14].

3.4. Clasificación multietiqueta con One-vs-Rest (OvR)

En multietiqueta cada request puede pertenecer a múltiples etiquetas. One-vs-Rest entrena un clasificador binario por etiqueta: para cada $\ell \in \{1, \dots, L\}$,

$$P(y_\ell = 1 \mid x) = \sigma(w_\ell^\top x + b_\ell). \quad (\uparrow)$$

Definición exacta:

- ℓ : índice de etiqueta (de 1 a L).
- $w_\ell \in \mathbb{R}^d$, $b_\ell \in \mathbb{R}$: parámetros del clasificador de la etiqueta ℓ .
- $y_\ell \in \{0, 1\}$: variable que indica presencia (1) o ausencia (0) de la etiqueta ℓ .

Para convertir probabilidades en predicción binaria se aplica un umbral (típicamente $\tau_\ell = 0,5$):

$$\hat{y}_\ell = \mathbb{I}[P(y_\ell = 1 \mid x) \geq \tau_\ell]. \quad - (\uparrow)$$

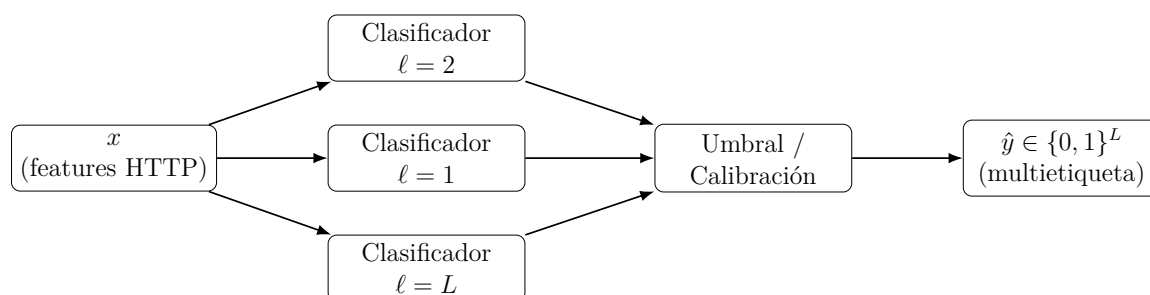


Figura 3.2: Esquema One-vs-Rest para multietiqueta: un clasificador binario por etiqueta [12, 11].

3.5. Métricas y base matemática

3.5.1. Binario

Para evaluación binaria definimos:

- **TP (true positives):** ataques/anómalos correctamente predichos como ataque.

- FP (false positives): normales predichos erróneamente como ataque.
- FN (false negatives): ataques predichos erróneamente como normal.
- TN (true negatives): normales correctamente predichos como normal.

Con estos conteos, las métricas ~~exportadas por los scripts~~ se calculan así:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right).$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \text{TPR} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

$$\text{Specificity} = \text{TNR} = \frac{TN}{TN + FP}, \quad \text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}, \quad \text{FNR} = \frac{FN}{FN + TP} = 1 - \text{Recall}.$$

Como medida balanceada entre las cuatro celdas de la matriz de confusión se reporta el *Matthews Correlation Coefficient* (MCC), recomendado como métrica robusta en escenarios desbalanceados [29]:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (1)$$

Cuando el modelo entrega un score continuo, también se calculan ROC-AUC y PR-AUC a partir de dicho score. En los scripts binarios esto se hace con `roc_auc_score(y_true, y_score)` y `average_precision_score(y_true, y_score)`. PR-AUC se conserva porque resulta más informativa que ROC-AUC cuando la clase positiva es minoritaria [28].

3.5.2. Multiclase: accuracy, promedios micro/macro/weighted y MCC

En multiclase, sea K la cantidad de clases y sea TP_k, FP_k, FN_k el conteo asociado a la clase k tratada en esquema *one-vs-rest*. En la implementación se reportan:

- **Accuracy:** proporción de instancias correctamente clasificadas.

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n \mathbb{I}[\hat{y}_i = y_i].$$

- **Balanced Accuracy:**  medio del recall por clase.

$$\text{Balanced Accuracy} = \frac{1}{K} \sum_{k=1}^K \frac{TP_k}{TP_k + FN_k}.$$

- **Precision, Recall y F_1 micro:** se calculan agregando primero todos los TP_k, FP_k y FN_k .

$$\text{Precision}_{\text{micro}} = \frac{\sum_k TP_k}{\sum_k (TP_k + FP_k)}, \quad \text{Recall}_{\text{micro}} = \frac{\sum_k TP_k}{\sum_k (TP_k + FN_k)}.$$

- **Precision, Recall y F_1 macro:** promedio simple de la métrica por clase.

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{Precision}_k, \quad F_{1,\text{macro}} = \frac{1}{K} \sum_{k=1}^K F_{1,k}.$$

- **Precision, Recall y F_1 weighted:** promedio por clase ponderado por su soporte real. Si s_k es la cantidad de ejemplos verdaderos de la clase k y $\sum_k s_k = n$,

$$F_{1,weighted} = \sum_{k=1}^K \frac{s_k}{n} F_{1,k}.$$

Los mismos pesos se usan para las versiones *weighted* de precision y recall.

- **MCC multiclase:** se calcula con la generalización implementada por `matthews_corrcoef`, adecuada para más de dos clases. (P.F. ^)

Si $K = 2$ y el modelo entrega probabilidades, los scripts supervisados agregan además ROC-AUC, PR-AUC, specificity, FPR, FNR y TPR. Si $K > 2$, exportan `roc_auc_ovr_macro`, `roc_auc_ovr_weighted`, `pr_auc_macro` y `pr_auc_weighted`, calculadas sobre una binarización one-vs-rest de las clases.

3.5.3. Multietiqueta: Hamming Loss, Jaccard y coincidencia exacta

En multietiqueta, $Y \in \{0, 1\}^{n \times L}$ y $\hat{Y} \in \{0, 1\}^{n \times L}$. Cada request puede activar ninguna, una o varias etiquetas.

Hamming Loss. Mide el error promedio por decisión etiqueta-por-request:

$$HL = \frac{1}{nL} \sum_{i=1}^n \sum_{\ell=1}^L \mathbb{I}[\hat{y}_{i\ell} \neq y_{i\ell}].$$

Un valor menor indica mejor desempeño.

Precision, Recall y F_1 micro/macro. Los scripts multilabel calculan versiones micro y macro de precision, recall y F_1 :

- **Micro:** agrega todas las decisiones positivas de todas las etiquetas antes de calcular la métrica.
- **Macro:** calcula la métrica por etiqueta y luego promedia en forma simple.

Jaccard Similarity. Para una request i , con conjuntos de etiquetas verdaderas Y_i y predichas $\hat{Y}_i$,

$$J_i = \frac{|Y_i \cap \hat{Y}_i|}{|Y_i \cup \hat{Y}_i|}.$$

En la implementación aparecen tres variantes, ~~según el script:~~

- **Jaccard micro:** agrega todos los aciertos y errores de todas las etiquetas.
- **Jaccard macro:** promedia el Jaccard por etiqueta.
- **Jaccard samples:** promedia J_i por instancia. ~~Esta variante aparece en el script multi-etiqueta dedicado cuando el problema no cae en modo binario reducido.~~

Exact Match Ratio. También llamado *subset accuracy*, mide la proporción de requests cuya predicción multilabel coincide exactamente con el conjunto verdadero:

$$\text{EMR} = \frac{1}{n} \sum_{i=1}^n \mathbb{I}[\hat{Y}_i = Y_i].$$

AUC multilabel. Cuando el script dispone de scores continuos por etiqueta, también calcula `pr_auc_micro`, `pr_auc_macro`, `roc_auc_micro` y `roc_auc_macro`. Además, en el análisis por etiqueta se repiten *precision*, *recall*, F_1 , Jaccard y, cuando corresponde, ROC-AUC y PR-AUC para cada CAPEC.

3.5.4. Métricas temporales y operativas

Además de las métricas predictivas, los scripts exportan o permiten reconstruir métricas operativas necesarias para discutir viabilidad.

- **Tiempo de optimización de parámetros:**

$$t_{opt} = t_{fin_busqueda} - t_{inicio_busqueda}.$$

En los bloques con `GridSearchCV`, `RandomizedSearchCV` o *halving*, coincide con el tiempo transcurrido de `search.fit(...)`; en el *one-class*, con el tiempo total de evaluación de candidatos sobre el *holdout* interno. Los scripts actuales ya miden este intervalo en consola y aquí se lo reporta como tiempo de optimización para mantener completo el bloque temporal del protocolo. Cuando una corrida se ejecuta con el ajuste de hiperparámetros desactivado, t_{opt} no corresponde a una medición válida y debe leerse como *no aplica*. En particular, la ablación multietiqueta con y sin tokens sospechosos fue implementada para ejecutar la variante ablacionada sin reoptimización adicional, salvo habilitación expresa de un re-ajuste. De ese modo, la diferencia entre la variante base y la variante ablacionada queda asociada a la retirada de ese grupo de variables y no a una nueva búsqueda de hiperparámetros. Si en algún cuadro de síntesis se fuerza un 0.00 para conservar la forma tabular, ese valor no describe un tiempo cronometrado nulo, sino la ausencia del bloque de búsqueda en esa corrida.

- **Tiempo de construcción del modelo:**

$$t_{build} = t_{fin_fit} - t_{ini_fit}.$$

En la implementación se exporta como `train_time_seconds`.

- **Tiempo de aplicación del modelo:** se mide en el benchmark como tiempo total del lazo de evaluación:

$$t_{apply} = \text{wall_time_seconds}.$$

Este valor se complementa con latencias por request para separar el costo de extracción de features y el costo de inferencia.

- ** Throughput:**

$$\text{Throughput} = \frac{\text{successful_requests}}{\text{wall_time_seconds}}.$$

- **Tasa de solicitudes fallidas del benchmark:** este indicador no describe un “fallo del modelo” ni un error de clasificación, sino la proporción de solicitudes que no pudieron completarse correctamente durante la medición operativa del benchmark.

$$\text{Tasa de solicitudes fallidas} = \frac{\text{failures}}{\text{total_requests_measured}}.$$

En todas las tablas operativas de este libro, esta métrica se interpreta como un indicador de estabilidad de la corrida de benchmark y no como una medida de desempeño predictivo.

- **Latencias total, de extracción y de inferencia:** cada request medida produce un tiempo total y, cuando corresponde, un desglose en extracción de features e inferencia. A partir de esos vectores se exportan media, desvío estándar y percentiles P50, P95 y P99.
- **CPU aproximada:**

$$\text{CPU util \%} \approx 100 \cdot \frac{\Delta(\text{user} + \text{system})}{\text{wall_time_seconds}}.$$

- **Memoria y tamaño de modelo:** se registran `rss_mb_before`, `rss_mb_after`, `peak_rss_mb_approx` y `model_size_bytes`.

3.5.5. Diccionario resumido de métricas retenidas

Métrica	Cálculo en la implementación	Uso principal
Accuracy	Proporción de predicciones correctas: $\frac{1}{n} \sum_i \mathbb{I}[\hat{y}_i = y_i]$.	Multiclase y casos reducidos a dos clases.
Balanced accuracy	Promedio del recall por clase; en binario equivale a $(\text{TPR} + \text{TNR})/2$.	Detección binaria y <i>one-class</i> , útil con desbalance.
Precision	$TP/(TP + FP)$ en binario; en agregaciones micro/macro/weighted sigue el promedio definido por el script.	Calidad de predicciones positivas.
Recall / TPR	$TP/(TP + FN)$. En binario coincide con la tasa de verdaderos positivos.	Capacidad de recuperación de ataques o clases reales.
F_1	Media armónica entre precision y recall.	Resumen sintético en detección binaria.
F_1 macro	Promedio simple del F_1 por clase o etiqueta.	Multiclase y multietiqueta, sensible a clases minoritarias.
F_1 weighted / micro	En multiclase se retiene F_1 ponderada por soporte; en multietiqueta se retiene F_1 micro, agregada globalmente.	Comparación global de modelos supervisados.
MCC	Coefficiente de correlación de Matthews; en multiclase se usa la generalización de <code>matthews_corrcoef</code> .	Resumen robusto con desbalance.
Hamming loss	Fracción de decisiones etiqueta-por-request incorrectas.	Multietiqueta.
Jaccard micro	Intersección sobre unión calculada globalmente sobre todas las etiquetas.	Multietiqueta.
Exact match ratio	Proporción de requests cuya salida multilabel coincide exactamente con la verdad.	Multietiqueta.

Métrica	Cálculo en la implementación	Uso principal
Tiempo de optimización	Tiempo total del bloque de búsqueda de hiperparámetros.	Tuning supervisado y <i>one-class</i> .
Tiempo de construcción	<code>train_time_seconds</code> , medido alrededor de <code>fit</code> .	Todos los modelos.
Tiempo de aplicación	<code>wall_time_seconds</code> del benchmark operativo.	Todos los modelos.

Capítulo 4

Datasets y criterios de selección

4.1. CAPEC como esquema semántico de ataques

CAPEC es un catálogo mantenido por MITRE que describe patrones de ataque (“cómo” se explota una debilidad), y se utiliza como taxonomía para clasificar ataques [3, 4].

4.2. SR-BH 2020: tráfico real de honeypot con etiquetas CAPEC

SR-BH 2020 fue recolectado en un servidor WordPress expuesto a Internet; su publicación y descripción técnica están disponibles en Harvard Dataverse (con DOI persistente) y en el artículo asociado [8, 9]. El dataset está alineado con un caso de uso WAF porque el registro se realiza a nivel de request HTTP [1, 2].

4.2.1. SR-BH 2020 como referencia de tráfico real

- **Tráfico real:** reduce el riesgo de sobre-ajuste a patrones sintéticos y captura variabilidad de producción [9].
- **Etiquetas CAPEC (multietiqueta):** permite ir más allá del binario y evaluar clasificación semántica por patrones [9, 4].
- **Pertinencia a WAF:** modela directamente campos inspeccionados por un WAF (método/URI/body) [1, 2].
- **Reproducibilidad:** al estar publicado con DOI persistente en Harvard Dataverse, facilita citación, trazabilidad y replicación de experimentos [8].

4.3. Datasets alternativos y unificación a un esquema común

Además de SR-BH, se contemplan datasets alternativos para comparar desempeño y *domain shift*:

- **CSIC 2010:** útil para benchmarking, pero su tráfico es generado/sintético [21].

- **TORPEDA (RECSI 2012)**: propone una especificación abierta de conjuntos de datos de ataques a aplicaciones web [22].

4.3.1. Estrategia de unificación (implementación)

Para poder entrenar/evaluar con el mismo pipeline, CSIC y TorpEda se transforman a un esquema compatible con SR-BH:

- **columnas**: `request_http_method`, `request_http_request`, `request_body`, y opcionalmente `request_headers_json`.
- **labels**: `label_binary`, `label_multiclass`, `label_multilabel`.

En todos los datasets, para el caso *one-class* se trata todo lo no-normal como anomalía (tarea **binaria**). Para las tareas supervisadas, la granularidad depende del dataset: **CSIC 2010** provee únicamente *normal/anómalo*, por lo que las configuraciones multiclase/multietiqueta se evalúan en su *versión reducida* (binaria); **TorpEda 2012** preserva sus etiquetas *multiclase* nativas (tipo de ataque) y además permite el colapso a binario cuando corresponda; **SR-BH 2020** habilita binario, multiclase (vía etiqueta CAPEC primaria) y multietiqueta CAPEC.

4.3.2. comparativa de la tríada de datasets

El diseño experimental conserva los tres datasets porque cada uno aporta una pieza distinta de evidencia metodológica. **SR-BH 2020** se usa como referencia principal de **realismo**, ya que contiene tráfico real de honeypot y etiquetas CAPEC multietiqueta [8, 9]. **CSIC 2010** y **TorpEda 2012** se mantienen porque siguen siendo puntos de comparación frecuentes en detección de ataques web, ofrecen escenarios más controlados y permiten contrastar cuánto del rendimiento observado depende del dataset y cuánto del modelo [21, 23, 22]. En otras palabras, la tríada se elige para cubrir simultáneamente **realismo**, **comparabilidad con la literatura** y **variación en granularidad de etiquetas**; quitar cualquiera de los tres debilitaría alguna de esas dimensiones.

4.3.3. Reducción multietiqueta a multiclase

En SR-BH una request puede tener múltiples etiquetas CAPEC. Para definir una tarea multiclase se colapsa la multietiqueta a una sola clase primaria, seleccionando la etiqueta con mayor severidad dentro del conjunto activo [5, 6]. Esta reducción es práctica para comparar modelos multiclase, pero implica pérdida de información y debe discutirse como amenaza a validez.

Capítulo 5

Representación de las peticiones HTTP

5.1. Principio general

La elección de features busca capturar:

- **Estructura:** profundidad del path, cantidad de parámetros, longitudes.
- **Codificación/anormalidad:** percent-encoding, proporción de caracteres no alfanuméricos.
- **Indicadores de payload:** presencia (como feature) de tokens comúnmente asociados a familias de ataque.
- **Método:** one-hot de métodos y bandera de método poco común.

En detección de anomalías web, atributos estadísticos y de estructura se han mostrado útiles para puntuar requests anómalos [10]. Además, trabajos más recientes sobre detección de ataques web y selección de variables muestran que señales extraídas directamente del request HTTP pueden capturar conocimiento experto útil sin abandonar la trazabilidad del modelo [25, 9]. Por ello, se priorizan features **baratas de extraer**, compatibles con la inspección de un WAF y suficientemente interpretables para discutir después qué señales aportan rendimiento real y cuáles se acercan demasiado a firmas [1, 2].

5.2. Lista de features implementada

La representación utilizada por los modelos se obtiene directamente a partir de cuatro insumos del request: el método HTTP, la URI, los encabezados y el cuerpo. En la implementación, la extracción se realiza sobre cada petición en forma individual. Primero se normaliza el método a mayúsculas, luego se descompone la URI con `urlsplit` para separar *path* y *query*, se procesan los parámetros con `parse_qs`, se inspecciona la presencia de secuencias de *percent-encoding* mediante una expresión regular y, finalmente, se revisan URI y body en minúsculas para contabilizar ocurrencias de tokens heurísticos. El resultado es un vector fijo de 25 variables numéricas y binarias.

5.2.1. Cómo se construye cada feature

Features de longitud

- **uri_len**: representa la longitud total de la URI original. Se calcula como la cantidad de caracteres de la cadena completa recibida en `uri`. Si la URI está vacía, el valor es 0.
- **query_len**: representa la longitud del *query string*. Después de separar la URI con `urlsplit`, se toma el componente `query` y se cuenta su cantidad de caracteres. Si la URI no tiene parámetros, el valor es 0.
- **body_len**: representa el tamaño del cuerpo del request en bytes. Se obtiene con `len(body)` cuando el cuerpo existe. Si el request no trae body, se asigna 0.
- **req_content_length**: representa el tamaño declarado del cuerpo. Se intenta leer primero el encabezado `Content-Length` (aceptando tanto `content-length` como `Content-Length`) y convertirlo a entero. Si el encabezado no existe, no es numérico o es negativo, se usa 0. Además, cuando ese valor queda en 0 pero el body existe, la implementación lo reemplaza por **body_len** para no perder completamente la señal de tamaño.

Features de estructura

- **path_depth**: aproxima la profundidad del recurso solicitado. Tras separar la URI, se toma el componente `path` y se cuenta cuántas veces aparece el carácter `/`. No mide niveles semánticos perfectos, pero sí ofrece una señal consistente sobre cuán profunda o anidada es la ruta.
- **n_query_params**: representa la cantidad de parámetros presentes en la query. Se calcula aplicando `parse_qsl(query, keep_blank_values=True)` y contando cuántos pares clave-valor fueron extraídos. Esto permite contabilizar también parámetros con valor vacío.
- **max_param_value_len**: representa la mayor longitud observada entre los valores de los parámetros de la query. Una vez obtenidos los pares clave-valor, se calcula el máximo de `len(valor)`. Si no hay parámetros, el valor es 0.

Features de composición de caracteres y codificación

- **uri_pct_non_alnum_ratio**: mide la proporción de caracteres no alfanuméricos dentro de la URI completa. Para obtenerla, se recorre la URI carácter por carácter, se cuenta cuántos no satisfacen `isalnum()`, y luego se divide esa cantidad por `uri_len`. Si la URI está vacía, el cociente se define como 0.
- **encoded**: aproxima el grado de *percent-encoding* en la URI. La implementación busca coincidencias del patrón `%[0-9a-fA-F]{2}`. Cada coincidencia representa una secuencia de tres caracteres codificados, por lo que se multiplica la cantidad de coincidencias por 3 y se divide por `uri_len`. El resultado es una razón entre 0 y 1 cuando la longitud es positiva.

- **body_encoded:** aplica la misma idea sobre el body. Si existe cuerpo, este se decodifica en UTF-8 ignorando errores; sobre ese texto se buscan las mismas secuencias de *percent-encoding*. Luego se divide la longitud total ocupada por esas coincidencias entre la longitud del texto decodificado. Si no hay body, el valor es 0.

Features heurísticas basadas en tokens sospechosos

En este estudio, los tokens sospechosos no se usan como reglas de bloqueo, sino como señales numéricas adicionales dentro de la representación. Se trata de una lista fija de cadenas elegida para capturar indicios frecuentes de traversal, *cross-site scripting*, inyección SQL y manipulación de payloads. La lista implementada es la siguiente:

```
../, <script, onerror=, onload=, ', ", --, ;,
union , select , sleep(, or 1=1, benchmark(, information_schema
```

Antes de contarlos, la URI y el body se convierten a minúsculas. Luego, para cada token de la lista, la implementación aplica una búsqueda por ocurrencias usando `count()` sobre la cadena correspondiente. En el código, cada token se normaliza con `strip().lower()` antes de contar, con lo cual los espacios laterales definidos en la lista no forman parte de la coincidencia final. Eso hace que la señal sea simple y reproducible, aunque también más sensible a apariciones parciales dentro del texto.

A partir de esa lista se construyen cuatro variables:

- **suspicious_tokens_count:** suma total de ocurrencias de todos los tokens heurísticos en la URI ya pasada a minúsculas.
- **has_suspicious_tokens:** indicador binario derivado de la variable anterior. Toma valor 1 cuando `suspicious_tokens_count` es mayor que 0, y 0 en caso contrario.
- **body_suspicious_tokens_count:** suma total de ocurrencias de los mismos tokens, pero en el body decodificado y pasado a minúsculas. Si el request no tiene body, el valor es 0.
- **body_has_suspicious_tokens:** indicador binario derivado de `body_suspicious_tokens_count`. Vale 1 cuando el conteo del body es mayor que 0, y 0 cuando no se detecta ninguna ocurrencia.

Estas variables se describen como heurísticas porque no provienen de un parser semántico ni de una taxonomía aprendida automáticamente. Son señales manuales, sencillas de calcular y cercanas a conocimiento experto del dominio web. Precisamente por eso, más adelante se consideran en forma crítica mediante ablaciones con y sin este grupo de variables.

Features del método HTTP

- **uncommon_method:** indica si el método observado cae fuera del conjunto más común {GET, POST, HEAD}. Después de convertir el método a mayúsculas, la variable toma valor 1 si pertenece a otro método distinto de esos tres; en caso contrario toma 0.

- **method_GET, method_POST, method_HEAD, method_PUT, method_DELETE, method_PATCH, method_OPTIONS, method_TRACE, method_CONNECT:** corresponden a una codificación one-hot del método HTTP. Cada una vale 1 únicamente cuando el método observado coincide exactamente con esa opción, y 0 en los demás casos.
- **method_OTHER:** complementa la codificación anterior. Vale 1 cuando el método no coincide con ninguno de los métodos conocidos contemplados por la implementación; si el método pertenece al conjunto conocido, esta variable vale 0.

5.2.2. Lectura metodológica de estas variables

El conjunto completo combina señales estructurales, de tamaño, de codificación, de contenido superficial y de protocolo. Algunas de ellas buscan capturar desvíos estadísticos del request, mientras que otras aproximan indicios más cercanos a firmas. Mantener esa mezcla dentro de un esquema fijo permite comparar modelos sobre una misma representación y, al mismo tiempo, analizar luego qué parte del rendimiento proviene de rasgos generales del tráfico HTTP y qué parte depende de heurísticas léxicas explícitas.

5.3. Criterios de diseño de la representación

- **Longitudes y estructura:** ataques suelen introducir payloads largos o patrones estructurales (query con muchos parámetros, paths profundos). Esto está alineado con enfoques basados en atributos del request para detección de anomalías y con trabajos de selección de features en tráfico web [10, 25].
- **Percent-encoding y no-alfanuméricos:** evasión y ofuscación frecuentemente se apoyan en *encoding* y caracteres especiales; modelarlos ayuda a capturar desviaciones que un WAF también inspecciona [1, 2].
- **Tokens sospechosos como features (no como reglas):** se usan como señales numéricas que pueden ayudar a modelos lineales/no lineales, pero deben discutirse críticamente porque se aproximan a firmas; por eso se justifican solo si luego se auditan con ablaciones [10, 25].
- **One-hot de métodos:** el método HTTP condiciona la forma del request; modelarlo evita confundir variaciones legítimas del protocolo con anomalías y mantiene coherencia con campos efectivamente inspeccionados en entornos WAF [1, 2].
- **req_content_length:** cuando está disponible, aporta una señal adicional de tamaño esperable del body con costo de extracción prácticamente nulo, útil para mantener compatibilidad entre datasets y enriquecer la descripción estructural del request [9, 1].

Capítulo 6

Metodología y diseño experimental

6.1. Tareas definidas

- **Detección binaria:** `label_binary` (0 normal, 1 ataque/anómalo).
- **Multiclase:** `label_multiclass`. En SR-BH: etiqueta CAPEC primaria derivada desde la multietiqueta con una política explícita; en TorpEda: clases nativas por tipo de ataque; en CSIC 2010 (sin taxonomía por tipo), se usa la versión reducida $K = 2$ (normal vs anómalo).
- **Multietiqueta:** `label_multilabel`. Disponible plenamente en SR-BH (vector/lista de etiquetas CAPEC por request). Para ejecutar el baseline OvR sobre los tres datasets, se parametriza la salida según etiquetas disponibles: CSIC 2010 se restringe a binario y TorpEda se usa en modo multiclase o binario cuando corresponda.

6.2. Matriz experimental (3 modelos $\times$ 3 datasets)

En términos operativos, el diseño experimental es una matriz completa: **cada familia de modelos se entrena y evalúa sobre cada uno de los tres datasets**, adaptando la granularidad de la tarea cuando el dataset no provee todas las etiquetas.

Cuadro 6.1: Matriz experimental (modelo $\times$ dataset) y tarea evaluada.

Modelo / configuración	CSIC 2010	TorpEda 2012	SR-BH 2020
Detector one-class escalable (SGDOneClassSVM + aproximación de kernel)	Binario (normal vs. anómalo)	Binario (normal vs. anómalo)	Binario (normal vs. anómalo)
Regresión logística lineal	Binario (versión reducida)	Multiclase (tipo de ataque)	Multiclase (CAPEC primaria)
OvR lineal	Binario (versión reducida)	Multiclase o binario, según etiqueta disponible	Multietiqueta (CAPEC)

6.3. Política general de particionado y reproducibilidad

La comparación entre modelos dentro de un mismo dataset debe hacerse sobre una **misma política de particionado**. Por ello, se adopta como protocolo principal un **particionado externo fijo 80/20** para los experimentos supervisados, con **semilla explícita y reproducible**. En esos casos, el **20 % externo** queda reservado exclusivamente para **prueba final**; no se reutiliza para ajuste de hiperparámetros, selección de variables, calibración de umbrales ni ajuste de transformaciones. En consecuencia, bajo este protocolo no hay contaminación entre entrenamiento y prueba: el conjunto de prueba externo permanece aislado hasta la evaluación final. Para el *detector one-class*, la política específica es entrenar con el **80 % de instancias normales** y evaluar sobre un conjunto mixto compuesto por el **20 % de normales en holdout** y el **total de anomalías/ataques disponibles**.

Como criterio adicional de trazabilidad:

- se registran las semillas, tamaños de partición y prevalencias de clases/etiquetas;
- cualquier transformación (`StandardScaler`, `MultiLabelBinarizer`, codificación u otras) se ajusta únicamente con datos de entrenamiento;
- cuando un dataset disponga de un *split* canónico propio (por ejemplo, CSIC 2010), este podrá reportarse como **análisis secundario**, pero la comparación principal entre modelos/datasets se basará en el protocolo 80/20 unificado.

6.4. Particionado por tarea

6.4.1. Supervisado (multiclase y multietiqueta)

En los modelos supervisados, el **80 % de entrenamiento** se utiliza para ajuste y selección de hiperparámetros; el **20 % externo** se usa una única vez para evaluación final.

Multiclase. Se intentará `train_test_split` estratificado por clase. Si alguna clase tiene cardinalidad insuficiente para sostener la estratificación o la validación cruzada, se aplicará una política explícita de clases raras previa al *split*: `keep`, `merge` (por ejemplo, a `OTHER_RARE`) o `drop`. Si aun así la estratificación no fuera viable, se utilizará un *split* no estratificado con semilla fija y se reportará la prevalencia final por clase.

Multietiqueta. Cuando sea técnicamente viable, se preferirá una estratificación aproximada que preserve la distribución conjunta de etiquetas. Si la esparsidad o las restricciones de implementación lo impiden, se utilizará como baseline un *split* fijo no estratificado y se reportarán las prevalencias por etiqueta antes y después del particionado. Esa condición se documenta como limitación y se complementa con análisis por etiqueta para evitar lecturas engañosas.

6.4.2. Detector one-class sin contaminación

Para el detector one-class, la política debe respetar dos condiciones: (i) entrenar solo con tráfico normal y (ii) mantener un conjunto de prueba verdaderamente no visto. Por ello, la estrategia adoptada es la siguiente:

1. Se realiza un **particionado 80/20 únicamente sobre las instancias normales**. El **20 % de normales en holdout** se reserva para prueba final.
2. El modelo final se ajusta *únicamente* con el **80 % de normales**. El **total de anomalías/ataques disponibles** se reserva para evaluación final y no participa del ajuste del modelo.
3. Si hay *tuning*, se realiza **solo dentro del bloque de entrenamiento normal**: se separa una validación interna de normales desde el 80 % de normales y se seleccionan hiperparámetros con un criterio coherente con el escenario *one-class* (por ejemplo, minimizar rechazo de tráfico normal o controlar una tasa objetivo de outliers), sin tocar el conjunto final de ataques.

Con ello se elimina el principal riesgo metodológico de este tipo de detector: usar el mismo conjunto mixto tanto para escoger hiperparámetros como para reportar métricas finales, y además se aprovechan **todas las anomalías/ataques disponibles** en la evaluación final.

6.5. Modelos seleccionados

Se mantienen las tres familias metodológicas definidas para el estudio, con una implementación ajustada para conservar compatibilidad con datasets muy grandes.

- **Detector one-class principal:** `StandardScaler + Nystroem + SGDOneClassSVM`. Se elige por su escalabilidad y porque preserva el supuesto de entrenar solo con tráfico normal [18, 19].
- **Baseline supervisado principal:** regresión logística lineal, preferentemente con `solver=saga` o configuración equivalente eficiente en gran escala, por su interpretabilidad, costo controlado y solidez como referencia supervisada [13, 14].
- **Baseline multietiqueta:** `OneVsRest(LogisticRegression)` con el mismo criterio de escalabilidad y reproducibilidad; se adopta porque OvR es una descomposición estándar y defendible cuando interesa conservar trazabilidad por etiqueta [12, 11, 14].

La lógica de selección no es “usar el modelo más complejo disponible”, sino elegir un conjunto de referencias que permita responder las preguntas del estudio con **trazabilidad, comparabilidad y factibilidad computacional**. Modelos más complejos, incluyendo enfoques recientes basados en redes profundas, son útiles como contexto de literatura, pero no necesariamente como *baseline* principal cuando el objetivo incluye interpretabilidad, evaluación cruzada entre datasets y ejecución sobre volúmenes del orden de 10^6 requests [26, 27].

Esta selección conserva las tres formulaciones analíticas definidas para el estudio y evita que el costo computacional del modelo y del extittuning invalide experimentalmente el protocolo.

6.6. Selección de hiperparámetros (Grid/Random Search) y validación

La metodología incorpora un módulo de *tuning* para alinear el pipeline con buenas prácticas experimentales, pero lo redefine para que no domine el costo total de ejecución. La política general es: **toda selección se hace solo dentro del bloque de entrenamiento** y con un **presupuesto computacional fijo** [16, 20].

6.6.1. Tuning en supervisado (multiclase y multietiqueta)

- **Bloque habilitado:** únicamente el **80 % de entrenamiento** del split externo.
- **Estrategia recomendada:** *random search* acotado o *successive halving* sobre un subconjunto interno del entrenamiento, evitando *grid search* exhaustivo como protocolo principal.
- **Parámetros buscados:** principalmente `C`, penalización, `solver` y, cuando corresponda, `class_weight`.
- **Métrica de selección:** F_1 weighted en multiclase y F_1 micro en multietiqueta.
- **Reentrenamiento final:** con la mejor configuración, el modelo se reentrena sobre todo el **80 % de entrenamiento** y se evalúa una única vez sobre el **20 % externo**.
- **Registro:** se guardan los scores por candidato a CSV para trazabilidad.

6.6.2. Tuning en el detector one-class

- **Bloque habilitado:** solo normales del **80 % de entrenamiento**; el test final nunca se toca durante la selección.
- **Estrategia recomendada:** búsqueda acotada sobre `n_components`, `gamma`, regularización y número de épocas del backend escalable.
- **Métrica de selección:** una métrica basada en normales del *holdout* interno (por ejemplo, tasa de aceptación de normales o tasa de rechazo falsa). Si se usa una variante con anomalías internas, debe declararse como análisis secundario y no como protocolo principal.
- **Reentrenamiento final:** con la mejor configuración, el modelo se reentrena sobre todo el **80 % de normales** y se evalúa una única vez sobre el **20 % de normales en holdout + todas las anomalías/ataques disponibles**.
- **Registro:** se guardan los scores por candidato a CSV para trazabilidad.

6.7. Configuración experimental efectivamente utilizada

Las corridas incorporadas en los resultados finales siguen una parametrización homogénea por familia de modelos. La regularidad entre datasets no se utilizó solo como criterio de

orden, sino como condición para sostener comparaciones transversales defendibles: cuando la tarea cambia, cambia la granularidad de la etiqueta; cuando la familia del modelo se mantiene, también se mantienen la lógica de particionado, el presupuesto de búsqueda y el bloque operativo de benchmark.

6.7.1. Construcción y preparación de los datasets

Los tres datasets se procesan sobre una representación común de features HTTP. En CSIC 2010 y CSIC/TorpEda 2012 se conserva el uso de información de cabeceras cuando está disponible, mientras que en SR-BH 2020 se mantiene el preprocesamiento compatible con separadores heterogéneos. El criterio adoptado es preservar un esquema de entrada comparable entre fuentes con distinto origen, sin introducir una definición distinta de features para cada dataset. De ese modo, las diferencias observadas en resultados quedan asociadas principalmente al realismo del tráfico, a la granularidad de las etiquetas y a la familia del modelo, y no a una representación de entrada completamente distinta.

6.7.2. Detector one-class en CSIC 2010, CSIC/TorpEda 2012 y SR-BH 2020

La familia *one-class* se ejecuta en los tres datasets como problema binario *normal vs. anómalo*, manteniendo una misma estructura: escalado, aproximación de kernel mediante Nyström y detector `SGDOneClassSVM`. El particionado conserva **80/20** sobre tráfico normal, con **semilla 42**, de modo que el ajuste se realiza exclusivamente con normales y la evaluación final utiliza el *holdout* de normales junto con el total de anomalías o ataques disponibles. La búsqueda de hiperparámetros se limita a un presupuesto aleatorio pequeño sobre cuatro ejes: fracción esperada de outliers, escala del kernel aproximado, dimensionalidad de la aproximación y número máximo de iteraciones.

Esa configuración cumple dos propósitos. En primer lugar, preserva el supuesto metodológico central del enfoque *one-class*: aprender la frontera del tráfico legítimo sin contaminar el entrenamiento con ataques. En segundo lugar, evita que el costo de explorar hiperparámetros vuelva impracticable una familia de modelos cuya principal justificación en el estudio es precisamente la escalabilidad. El bloque operativo también se mantiene uniforme, con un benchmark acotado y repetición mínima, suficiente para medir latencia, *throughput*, memoria y estabilidad sin convertir la evaluación operativa en una prueba de carga ajena al alcance definido.

Además, en los tres datasets se ejecutan dos variantes: una base y otra sin variables de tokens sospechosos. La ablación mantiene intacto el resto del pipeline. Esa simetría permite interpretar cualquier diferencia de rendimiento como efecto del grupo de variables removido y no como efecto colateral de otra modificación simultánea.

6.7.3. Regresión logística multiclase en CSIC 2010, CSIC/TorpEda 2012 y SR-BH 2020

La familia multiclase se ejecuta con regresión logística lineal, `solver=lbfgs`, penalización 12, regularización inicial `C=1.0`, ponderación de clases `balanced`, máximo de 500 iteraciones y política explícita de clases raras basada en `keep` con mínimo de dos instancias. Sobre esa base fija, la búsqueda de hiperparámetros se restringe a una validación cruzada

de tres particiones, presupuesto aleatorio 2 y una muestra interna acotada, explorando únicamente dos valores de regularización y dos alternativas de ponderación de clases.

La justificación de esta configuración no reside en exhaustividad, sino en estabilidad. `lbfgs` con `l2` conserva una línea base lineal robusta, con convergencia previsible y lectura posterior por coeficientes. La exploración se concentra en regularización y ponderación de clases porque, dentro de una familia lineal de baja complejidad, esos son los dos ejes con mayor impacto práctico bajo desbalance. Fijar el optimizador y la penalización reduce el espacio de búsqueda, disminuye ruido experimental y hace más clara la comparación entre datasets.

En SR-BH 2020 la tarea multiclase se apoya en la etiqueta CAPEC primaria derivada desde la anotación multietiqueta. En CSIC/TorpEda 2012 se conserva la taxonomía nativa por tipo de ataque. En CSIC 2010 la misma tubería se mantiene, pero la salida queda reducida de hecho a un problema binario, ya que el dataset no aporta una taxonomía multiclase rica. Esa configuración no contradice la matriz experimental: muestra, por el contrario, qué ocurre cuando una misma familia supervisada se aplica a datasets con distinta granularidad semántica.

También aquí se conservan dos variantes, base y sin tokens sospechosos, bajo el mismo particionado externo, el mismo presupuesto de *tuning* y el mismo benchmark operativo. Esa regularidad refuerza la trazabilidad del contraste.

6.7.4. OvR lineal para multietiqueta y modos reducidos por dataset

La familia multietiqueta se ejecuta mediante una descomposición *One-vs-Rest* sobre regresión logística lineal con `solver=lbfgs`, penalización `l2`, `class_weight=balanced`, máximo de 300 iteraciones, tolerancia `1e-3` y paralelismo habilitado en el entrenamiento de las salidas. La búsqueda de hiperparámetros mantiene el mismo criterio acotado que en multiclase: validación cruzada interna, presupuesto 2, muestra interna limitada y exploración restringida a regularización y ponderación de clases.

La formulación plena multietiqueta se conserva en SR-BH 2020, que sí ofrece etiquetas CAPEC simultáneas por request. En CSIC 2010 se fuerza un modo binario reducido, porque el dataset no dispone de una salida multietiqueta real. En CSIC/TorpEda 2012 se utiliza un modo reducido automático, que adapta la salida a la granularidad efectivamente disponible. El esquema conserva una única familia de script y de evaluación para los tres datasets, pero sin atribuir una riqueza semántica que dos de ellos no poseen. De esa manera, la comparación conserva continuidad operacional sin introducir etiquetas artificiales.

La ablación de tokens sospechosos en esta familia se ejecuta dentro del mismo flujo del modelo y se registra como comparación específica entre variante base y variante ablacionada. El procedimiento evita duplicar lógica de entrenamiento y mantiene alineadas las métricas exportadas para ambas variantes. En el bloque operativo, el benchmark utiliza una ventana algo más amplia de calentamiento que en multiclase, porque la descomposición por etiqueta introduce un costo superior de inferencia y de carga en memoria.

6.7.5. Lectura metodológica conjunta

En conjunto, la forma de ejecución de cada familia responde a un mismo criterio: **máxima comparabilidad con complejidad controlada**. El detector *one-class* conserva su supuesto de entrenamiento exclusivo con normales; la regresión logística multiclase

mantiene una línea base lineal estable y fácilmente interpretable; y la formulación OvR multietiqueta preserva trazabilidad por etiqueta sin introducir arquitecturas más pesadas que desplacen la discusión central. Los presupuestos de *tuning* son pequeños en los tres casos, no porque la optimización carezca de valor, sino porque la prioridad metodológica del estudio consiste en sostener una matriz experimental reproducible, coherente con el hardware disponible y suficiente para responder las preguntas de comparación planteadas.

6.8. Métricas (efectividad y eficiencia)

Dado que uno de los objetivos es comparar con trabajos relacionados, se selecciona un conjunto de métricas recurrente en la literatura sobre detección y clasificación de ataques web. El criterio de selección de métricas parte de que **accuracy por sí sola no alcanza** en presencia de desbalance, clases raras y tareas distintas (binaria, multiclase y multietiqueta). Por eso se combinan métricas de rendimiento, métricas sensibles al tipo de error y métricas operativas de costo temporal [28, 29, 9, 12].

- **Binario (normal vs ataque/anómalo):** matriz de confusión, Accuracy, Balanced Accuracy, Precision, Recall/TPR, F_1 , Specificity/TNR, FPR/FNR, MCC, ROC-AUC y PR-AUC. PR-AUC se incluye porque es más informativa que ROC-AUC cuando la clase positiva es minoritaria [28].
- **Multiclase:** Accuracy, Balanced Accuracy, Precision/Recall/ F_1 en agregación micro, macro y *weighted*, además de MCC, matriz de confusión y métricas por clase. Se reportan varias agregaciones para no ocultar el comportamiento sobre clases raras detrás de un único promedio [12, 27].
- **Multietiqueta (CAPEC):** Hamming Loss, Jaccard Similarity (micro, macro y, cuando el script lo habilita, por muestra), Exact Match Ratio, Precision/Recall/ F_1 micro y macro, y AUC micro/macro cuando se dispone de scores continuos. Estas métricas cubren error por etiqueta, solapamiento entre conjuntos y coincidencia exacta de la predicción multilabel [9, 12].
- **Robustez al desbalance:** MCC como métrica complementaria a accuracy/ F_1 , ya que resume de forma más equilibrada las cuatro celdas de la matriz de confusión [29].
- **Costo temporal del protocolo:** se reportan explícitamente el **tiempo de optimización de parámetros**, el **tiempo de construcción del modelo** y el **tiempo de aplicación del modelo**. El primero corresponde al tiempo total del bloque de búsqueda de hiperparámetros; el segundo, al tiempo medido alrededor de `fit` y exportado como `train_time_seconds`; y el tercero, al tiempo total del benchmark operativo, exportado como `wall_time_seconds` y complementado con latencias por request.
- **Eficiencia/tiempo real:** latencia media y percentiles P50/P95/P99 del pipeline (extracción de features + inferencia), throughput (req/s), tasa de fallos, CPU/RAM, tamaño del modelo y desagregación entre extracción e inferencia. Estas métricas se incluyen porque la evaluación no solo considera efectividad, sino también viabilidad operativa en un entorno tipo WAF [24].

El detalle formal de cada métrica y de su cálculo exacto dentro de la implementación se resume en la Sección 3.5.

6.9. Criterios de comparación con la literatura

Comparar resultados entre papers puede ser engañoso si difieren *splits*, preprocesamiento o definición de etiquetas. Para maximizar comparabilidad:

- se reporta el **mismo conjunto de métricas** usado con frecuencia en la literatura;
- cuando un paper usa el mismo dataset y tarea, se replica la tarea y se reportan métricas homólogas;
- cuando un paper usa una configuración distinta, se compara solo a nivel de **métrica comparable** y se documenta explícitamente la amenaza a validez.

Cuadro 6.2: Métricas reportadas en trabajos relacionados y su correspondencia con el estudio.

Trabajo	Métricas típicas reportadas
WAF basado en detección de anomalías	TPR/FPR, F1 y medición de overhead en tiempo de respuesta (ms) [24].
SR-BH (paper del dataset)	Multietiqueta: Hamming Loss, Jaccard, EMR, F1 y AUC [9].
Selección de features en web attacks (Riv-rol 2024)	Accuracy, TPR, FPR y AUC/ROC (curvas ROC) [25].
Modelos recientes con énfasis en IDS (Fang 2024)	Accuracy, Precision, Recall, F1, FPR/FNR y MCC [26, 29].
Ensamble/Deep Kernel Learning (Zhou 2024)	Accuracy, macro Precision/Recall/F1 y weighted Precision/F1 (desbalance) [27].

6.10. Importancia de características (FI)

Esta sección se incluye porque la comparación no se limita a contrastar métricas finales entre modelos. El diseño experimental también requiere examinar el aporte relativo de los grupos de características utilizados y discutir hasta qué punto el rendimiento observado proviene de señales estructurales del request HTTP o de indicadores léxicos más cercanos a firmas. Por esa razón, además de medir desempeño, el protocolo incorpora un bloque específico para analizar **qué variables empujan la decisión del modelo y qué significado metodológico tiene ese aporte**.

En este contexto, **interpretabilidad** se entiende como la posibilidad de relacionar el comportamiento del modelo con variables concretas de entrada. En modelos lineales, esto significa poder revisar qué features reciben mayor peso y en qué dirección contribuyen a la predicción. **Explicabilidad**, en un sentido más amplio, refiere a la capacidad de justificar por qué un modelo rinde como rinde y qué tipo de señal está aprovechando. Aunque ambos términos suelen usarse juntos, aquí conviene distinguirlos: la interpretabilidad apunta a leer el modelo; la explicabilidad apunta a sostener una discusión metodológica sobre sus decisiones y sus límites.

La sigla **FI** (*Feature Importance*) se usa como nombre corto para el análisis de importancia de variables. Aquí no se plantea como una etapa separada y costosa, sino como

una lectura complementaria del mismo protocolo experimental. La idea no es construir un ranking absoluto y definitivo de variables, sino observar si ciertos grupos —por ejemplo, longitudes, estructura de URI, método HTTP o tokens sospechosos— sostienen una parte importante del rendimiento.

La estrategia adoptada tiene tres niveles:

- **Coeficientes estandarizados (LR/OvR):** como los modelos supervisados son lineales y se entrenan sobre variables escaladas, la magnitud de los coeficientes permite comparar de forma razonable el peso relativo de cada feature. En multiclase, la lectura se hace por clase; en multietiqueta, por etiqueta.
- **Ablaciones por grupos:** se comparan corridas del pipeline completo contra variantes en las que se elimina un grupo entero de variables. Esto permite discutir el aporte del grupo como bloque y no solo de features aisladas.
- **Ablación focal con y sin tokens sospechosos:** se reserva una comparación específica para ese grupo porque es el más delicado metodológicamente. Si gran parte del rendimiento dependiera de esas variables, la representación quedaría más cerca de una lógica de firma que de una descripción general del tráfico HTTP.

La sección está, entonces, para sostener una pregunta central del estudio: no solo cuál modelo rinde mejor, sino también **por qué** rinde mejor y **sobre qué tipo de señal** lo hace. Esa discusión resulta importante para interpretar resultados, comparar con la literatura y valorar la factibilidad de usar la representación adoptada en un contexto tipo WAF sin convertir el pipeline en un conjunto encubierto de reglas manuales.

6.11. Amenazas a la validez

- **Desbalance:** etiquetas CAPEC pueden ser raras; afecta F1 macro, estratificación y estabilidad del entrenamiento.
- **Reducción multilabel→multiclase:** seleccionar una única etiqueta primaria simplifica la tarea, pero colapsa información multietiqueta [5, 6, 7].
- **Evaluación one-class:** si el conjunto mixto de prueba se reutiliza para tuning, las métricas se inflan; por eso el protocolo separa la validación interna basada en normales del conjunto final de prueba compuesto por holdout de normales y todas las anomalías/ataques disponibles.
- **Heurísticas de tokens:** pueden sesgar los modelos hacia firmas; por eso se exige la ablación correspondiente.
- **Domain shift:** datasets sintéticos/curados (CSIC/TorpEda) pueden no transferir a tráfico real (SR-BH).
- **Validez externa:** aun en SR-BH, el tráfico proviene de un honeypot específico y no de todas las configuraciones de producción posibles.

Capítulo 7

Implementación (pipeline)

7.1. Arquitectura del pipeline

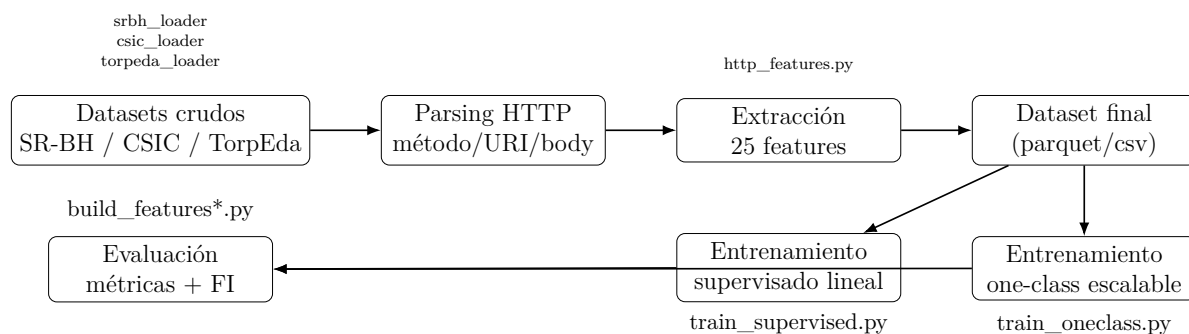


Figura 7.1: Pipeline propuesto: unificación de datasets, extracción de features, entrenamiento escalable, tuning con presupuesto fijo y evaluación.

7.2. Artefactos generados

7.3. Derivación de `label_multiclass` a partir de CAPEC (SR-BH)

En SR-BH, una misma request puede activar varias etiquetas CAPEC al mismo tiempo. Esa es precisamente la razón por la que el dataset resulta valioso para estudiar una formulación multietiqueta. Sin embargo, para poder compararlo con tareas multiclase sobre un mismo pipeline, la implementación también necesita una única etiqueta principal por instancia.

La reducción a multiclase se resuelve mediante una regla determinista basada en severidad. Primero se construye `label_multilabel` como la lista de etiquetas CAPEC activas en cada fila. Luego, cuando la estrategia seleccionada es `severity`, se elige como `label_multiclass` la etiqueta con mayor severidad según el mapa definido en el loader. Si no hay etiquetas activas, la instancia queda como `NORMAL`. Esta decisión no elimina la salida multietiqueta original: ambas conviven para que la comparación entre formulaciones no obligue a perder información semántica.

La implementación conserva además una columna auxiliar `label_multiclass_first`, útil para auditoría, y una columna `label_primary_severity` que deja trazado el puntaje

de severidad asociado a la etiqueta elegida. De esta manera, la reducción a multiclase no queda como una operación opaca, sino como una transformación explícita y revisable dentro del mismo dataset procesado.

Desde el punto de vista metodológico, esta política se usa solo para habilitar una tarea adicional de comparación. La salida multietiqueta sigue siendo la representación más fiel del fenómeno cuando una misma request participa de más de un patrón CAPEC. Por eso, en la discusión de resultados, la lectura multiclase se interpreta como una simplificación útil para contraste experimental, no como sustituto de la anotación original.

Capítulo 8

Resultados

8.1. Criterio de presentación

El capítulo de resultados adopta una organización estrictamente tabular por tarea, dataset y variante de modelo. La decisión responde a dos necesidades simultáneas. Por un lado, resulta necesario comparar enfoques *one-class*, multiclase y multietiqueta sobre CSIC 2010, CSIC/TorpEda 2012 y SR-BH 2020, con métricas adecuadas a cada formulación y con una discusión explícita de factibilidad tipo WAF. Por otro lado, el propio protocolo metodológico del estudio establece que la comparación debe mantener separados entrenamiento, ajuste y prueba, y que además del rendimiento predictivo deben informarse tiempos, latencias, *throughput*, consumo de recursos y tamaño del modelo.

Para preservar comparabilidad entre artefactos heterogéneos, este capítulo distingue dos familias de tablas. Las primeras reúnen las métricas de efectividad reportadas por cada corrida. Las segundas reúnen las métricas operativas. En los bloques operativos, el tiempo de aplicación $t_{apply}^\dagger$ se expresa como el tiempo aproximado del lote principal de benchmark, calculado como *solicitudes medidas / throughput global*; esta definición permite homogeneizar artefactos que no siempre exportan el mismo nombre de campo para el tiempo de pared del benchmark, pero sí informan de manera estable el volumen medido y el *throughput*.

8.2. Lectura metodológica de las corridas

Las corridas incorporadas en este libro siguen una lógica metodológica común: una representación compartida de peticiones HTTP, tres datasets con distinta granularidad de etiquetado y modelos compatibles con cada formulación del problema. CSIC 2010 se conserva como referencia controlada, útil para contrastar normalidad frente a tráfico anómalo y para observar qué ocurre cuando una tarea declarada como multiclase o multietiqueta se reduce en la práctica a un problema mucho más simple. CSIC/TorpEda 2012 aporta tipologías explícitas de ataque y un escenario genuinamente multiclase. SR-BH 2020, derivado de tráfico real de *honeypot*, conserva el valor central de aportar mayor realismo y una semántica CAPEC más rica que la de los datasets puramente sintéticos o de laboratorio [8, 9, 21, 23, 22].

La selección de modelos también sigue esa lógica de adecuación a la tarea y de costo computacional defendible. Para detección binaria de anomalías se utilizó un esquema `StandardScaler + Nystroem + SGDOneClassSVM`, porque permite aproximar una frontera no lineal de normalidad con costo mucho menor que el de un kernel exacto, manteniendo

el supuesto central de entrenar solo con tráfico legítimo [30, 31, 18, 19]. Para multiclase y multietiqueta se adoptó regresión logística lineal, y en el caso multietiqueta se la desplegó en esquema One-vs-Rest. La elección es coherente con los objetivos del estudio: interpretabilidad, trazabilidad, bajo costo de inferencia y una línea base suficientemente fuerte para comparar formulaciones sin que la complejidad del modelo eclipse la discusión experimental [13, 14, 11, 12].

El ajuste de hiperparámetros se mantuvo acotado. En los modelos supervisados se usó búsqueda aleatoria con presupuestos muy pequeños y validación cruzada interna, mientras que en el detector one-class se restringió la exploración a parámetros que controlan la forma de la frontera, la aproximación del kernel y la convergencia. No se presenta una optimización exhaustiva, sino una sintonización ligera y reproducible, suficiente para evitar configuraciones arbitrarias sin convertir el tuning en el costo dominante del protocolo [32, 16].

La ablación de variables basadas en *suspicious tokens* se mantuvo en los tres bloques porque cumple una función metodológica importante: separar el aporte de señales estructurales y de codificación de aquellas señales que se aproximan más a firmas léxicas explícitas. Cuando la ablación apenas modifica los resultados, puede sostenerse que el modelo depende poco de esos atajos. Cuando la ablación degrada fuertemente el rendimiento, la interpretación correcta no es que el modelo sea “malo, sino que parte de su capacidad discriminativa descansa efectivamente en conocimiento léxico de dominio, algo que conviene declarar para no sobredimensionar la supuesta neutralidad de la representación.

8.3. Detección one-class

8.3.1. Métricas de efectividad

Cuadro 8.1: Resultados de detección one-class por dataset y variante.

Dataset	Variante	Acc.	Bal. Acc.	Prec.	Recall	F1	MCC	TNR	FPR	FNR	ROC AUC	PR AUC
CSIC 2010	Base	0.6951	0.7512	0.9581	0.5437	0.6938	0.5037	0.9586	0.0414	0.4563	0.7688	0.8801
CSIC 2010	Sin tokens	0.7013	0.7631	0.9916	0.5341	0.6943	0.5340	0.9922	0.0078	0.4659	0.7630	0.8762
SR-BH 2020	Base	0.3175	0.5617	0.9813	0.1327	0.2338	0.1648	0.9908	0.0092	0.8673	0.5158	0.8547
SR-BH 2020	Sin tokens	0.3124	0.5585	0.9804	0.1261	0.2235	0.1596	0.9908	0.0092	0.8739	0.4736	0.8276
CSIC/TorpEda 2012	Base	0.7440	0.8615	0.9995	0.7379	0.8490	0.2504	0.9851	0.0149	0.2621	0.8358	0.9954
CSIC/TorpEda 2012	Sin tokens	0.7369	0.7661	0.9930	0.7354	0.8450	0.1848	0.7968	0.2032	0.2646	0.8215	0.9949

8.3.2. Métricas operativas

Cuadro 8.2: Indicadores operativos de las corridas one-class.

Dataset	Variante	t_{opt} (s)	t_{build} (s)	$t_{apply}^\dagger$ (s)	Thr.	P95 (ms)	CPU (%)	Peak RSS (MB)	Modelo (KiB)
CSIC 2010	Base	4.120	1.071	2.076	192.708	9.916	130.56	873.45	569.5
CSIC 2010	Sin tokens	4.264	1.161	2.429	164.639	10.697	144.47	861.43	561.2
SR-BH 2020	Base	3.859	5.270	1.629	245.480	6.032	123.97	5448.73	569.5
SR-BH 2020	Sin tokens	5.142	6.797	1.648	242.650	5.458	124.36	5429.17	561.2
CSIC/TorpEda 2012	Base	1.330	0.380	1.611	248.260	5.135	146.47	942.27	569.5
CSIC/TorpEda 2012	Sin tokens	1.238	0.249	1.469	272.250	4.518	159.26	902.93	561.2

En conjunto, las corridas one-class muestran un patrón estable. La variante base conserva la mejor lectura global en Harvard y en TorpEda, especialmente por su mejor *balanced accuracy*, MCC y control del error sobre la clase normal. En CSIC 2010, en

cambio, la ablación mejora levemente Accuracy, Balanced Accuracy y MCC, aunque lo hace a costa de una caída pequeña en ROC AUC y PR AUC. Estos resultados indican que el bloque one-class se beneficia de una representación no lineal suficientemente rica como para no depender de manera uniforme de los tokens sospechosos.

Operativamente, el costo de entrenamiento final del detector es bajo en los tres datasets, pero el costo de memoria no lo es. Harvard es, con mucha diferencia, el caso más exigente: supera los 5.4 GB de pico de RSS en ambas variantes y, aunque mantiene *throughput* alto, lo hace con una huella de memoria muy superior al resto. TorpEda ofrece la mejor combinación entre discriminación y costo. CSIC queda en una posición intermedia, con resultados predictivos razonables y tiempos compatibles con una discusión de viabilidad, aunque sin la solidez de TorpEda en separación binaria.

8.4. Clasificación multiclase

8.4.1. Métricas de efectividad

Cuadro 8.3: Resultados multiclase por dataset y variante. En CSIC 2010 las AUC corresponden a la formulación binaria efectiva observada en prueba; en Harvard y TorpEda corresponden a la lectura OvR ponderada reportada por los artefactos.

Dataset	Variante	Acc.	Bal. Acc.	F1 macro	F1 weighted	Prec. macro	Recall macro	MCC	ROC AUC	PR AUC
CSIC 2010	Base	0.8286	0.6989	0.7291	0.8085	0.8248	0.6989	0.5084	0.8430	0.9229
CSIC 2010	Sin tokens	0.8294	0.7011	0.7313	0.8098	0.8249	0.7011	0.5112	0.8439	0.9226
SR-BH 2020	Base	0.8014	0.5039	0.5252	0.7854	0.6240	0.5039	0.6450	0.9363	0.8631
SR-BH 2020	Sin tokens	0.7173	0.3426	0.3837	0.6882	0.5388	0.3426	0.4875	0.8958	0.8003
CSIC/TorpEda 2012	Base	0.8408	0.2916	0.3011	0.8092	0.5303	0.2916	0.7384	0.9675	0.8499
CSIC/TorpEda 2012	Sin tokens	0.8136	0.2556	0.2531	0.7747	0.3251	0.2556	0.6900	0.9569	0.8114

8.4.2. Métricas operativas

Cuadro 8.4: Indicadores operativos de las corridas multiclase.

Dataset	Variante	t_{opt} (s)	t_{build} (s)	$t_{apply}^{\dagger}$ (s)	Thr.	P95 (ms)	CPU (%)	Peak RSS (MB)	Modelo (KiB)
CSIC 2010	Base	2.4525	1.0872	1.5929	376.6683	3.2893	114.26	479.48	3.42
CSIC 2010	Sin tokens	2.3335	1.0055	1.4508	413.5759	2.6209	118.56	467.23	3.08
SR-BH 2020	Base	10.0885	137.3066	1.5420	388.8543	2.9597	119.90	2380.93	6.87
SR-BH 2020	Sin tokens	9.5461	144.5713	1.5916	376.9847	3.1997	118.12	2269.09	6.12
CSIC/TorpEda 2012	Base	3.8222	14.6148	1.4965	400.9321	2.8933	118.28	478.72	5.51
CSIC/TorpEda 2012	Sin tokens	3.7528	12.0153	1.5008	399.7902	2.8655	116.61	472.93	4.88

Las tres corridas multiclase dejan una señal metodológica nítida: el efecto de la ablación depende fuertemente del dataset. En CSIC 2010, donde la prueba termina comportándose como una discriminación de dos etiquetas efectivas, la diferencia entre variantes es mínima y la versión sin tokens obtiene una mejora marginal en Accuracy, Balanced Accuracy, F1 macro y MCC. En Harvard la situación es la opuesta: la ablación degrada todas las métricas globales de manera marcada, lo que sugiere que las señales léxicas aportan información importante para separar varias tipologías de ataque sobre un espacio de clases amplio y heterogéneo. En TorpEda el resultado también favorece al modelo base, sobre todo en macro F1, MCC y AUC.

La lectura por clases, analizada en los artefactos específicos de cada corrida, muestra además que la degradación no se distribuye de forma pareja. En Harvard, por ejemplo, el impacto negativo de quitar tokens sospechosos es especialmente visible en clases como

CAPEC-242, CAPEC-34 y CAPEC-194, mientras que otras categorías prácticamente no cambian. Eso refuerza una conclusión importante para el libro: la utilidad de las señales léxicas no es uniforme ni universal, pero tampoco puede tratarse como un simple adorno.

En el plano operativo, multiclase es el bloque más liviano en inferencia para CSIC y TorpEda, con latencias P95 por debajo de 3.3 ms y *throughput* cercano o superior a 400 req/s. Harvard mantiene una inferencia rápida, pero el costo de construcción del modelo es muy superior al de los otros datasets y su pico de memoria durante benchmark también crece con fuerza. Por lo tanto, la discusión de viabilidad debe separar claramente costo de entrenamiento y costo de aplicación: Harvard multiclase es operativo una vez entrenado, pero no es el escenario más cómodo para reentrenamientos frecuentes.

8.5. Clasificación multietiqueta

8.5.1. Métricas globales

Cuadro 8.5: Métricas globales multietiqueta por dataset y variante.

Dataset	Variante	F1 micro	F1 macro	Hamming	Jaccard micro	EMR	t_{build} (s)	$t_{apply}^{\dagger}$ (s)	Thr.
CSIC 2010	Base	0.8336	0.7361	0.1664	0.7147	0.8336	2.4590	2.0293	295.67
CSIC 2010	Sin tokens	0.7771	0.7338	0.2229	0.6355	0.7771	1.9575	1.9460	308.33
SR-BH 2020	Base	0.6582	0.1079	0.0039	0.4906	0.7820	19.96	6.57	91.32
SR-BH 2020	Sin tokens	0.2052	0.1013	0.0508	0.1143	0.0723	37.69	6.36	94.29
CSIC/TorpEda 2012	Base	0.8498	0.2601	0.0288	0.7388	0.7681	2.6578	2.2321	268.84
CSIC/TorpEda 2012	Sin tokens	0.4725	0.2393	0.2085	0.3094	0.3902	1.5397	2.3763	252.49

Cuadro 8.6: Métricas extendidas disponibles para las corridas multietiqueta de Harvard y TorpEda.

Dataset	Variante	Prec. micro	Recall micro	PR AUC micro	PR AUC macro	ROC AUC micro	ROC AUC macro
SR-BH 2020	Base	0.9376	0.5071	0.7297	0.1383	0.9903	0.7075
SR-BH 2020	Sin tokens	0.1160	0.8886	0.1271	0.1269	0.9718	0.9571
CSIC/TorpEda 2012	Base	0.8746	0.8263	0.9244	—	0.9864	—
CSIC/TorpEda 2012	Sin tokens	0.3148	0.9473	0.7721	—	0.9555	—

El Cuadro 8.6 requiere una aclaración explícita. En SR-BH 2020, la variante base conserva la mejor combinación entre precisión micro, *PR AUC* micro y *ROC AUC* micro, mientras que la ablación sin tokens sospechosos empuja el *recall* micro a valores muy altos a costa de una caída extrema de precisión. Esa combinación es típica de una sobrepredicción agresiva: el modelo recupera más etiquetas positivas, pero también dispara muchos más falsos positivos. El hecho de que *ROC AUC* macro resulte mayor en la ablación no contradice esa lectura, porque AUC resume capacidad de ordenamiento sobre distintos umbrales, mientras que precisión, *exact match*, Jaccard y F_1 dependen de la decisión binaria finalmente aplicada en inferencia.

En CSIC/TorpEda 2012, los símbolos — de *PR AUC* macro y *ROC AUC* macro no representan rendimiento nulo ni error de cálculo. Responden a una decisión metodológica ligada a la forma en que esas corridas fueron formuladas. El script multietiqueta incorpora un modo de compatibilidad **reduced-binary-mode** para datasets cuya columna multietiqueta queda efectivamente reducida a una única dimensión positiva; en ese escenario el problema deja de comportarse como un multietiqueta rico y pasa a interpretarse como un caso binario reducido, con bloque específico de métricas binarias y con menor utilidad

real de los promedios entre etiquetas. En los comandos utilizados, esa lógica se activa de manera forzada para CSIC y se habilita en modo automático para TorpEda. Bajo esa formulación, los promedios macro de *PR AUC* y *ROC AUC* dejan de aportar una lectura sustantivamente distinta: con una sola dimensión efectiva, el promedio macro colapsa hacia la misma señal de ordenamiento que ya queda representada por la evaluación binaria del objetivo activo y por las métricas micro conservadas en el cuadro. Por esa razón se prefirió marcar esas celdas con — en lugar de exhibirlas como si fueran evidencia adicional de un comportamiento multietiqueta pleno, algo que habría sobreinterpretado la estructura real de esas corridas.

Cuadro 8.7: Métricas binarias derivadas para CSIC 2010 en su formulación multietiqueta reducida a binario.

Variante	Acc.	Bal. Acc.	Prec.+	Recall+	F1+	MCC	TNR	FPR	FNR
Base	0.8336	0.7046	0.8353	0.4392	0.5757	0.5231	0.9700	0.0300	0.5608
Sin tokens	0.7771	0.7608	0.5502	0.7272	0.6264	0.4812	0.7944	0.2056	0.2728

Cuadro 8.8: Indicadores operativos consolidados de las corridas multietiqueta.

Dataset	Variante	P95 (ms)	CPU (%)	Peak RSS (MB)	Modelo (KiB)	Tasa de solicitudes fallidas	Lectura
CSIC 2010	Base	4.2743	132.07	445.46	4.17	0.0000	mejor efectividad global
CSIC 2010	Sin tokens	4.2951	122.30	445.56	3.85	0.0000	más rápido bajo carga leve
SR-BH 2020	Base	13.60	122.52	2862.48	50.76	0.0000	mejor equilibrio semántico
SR-BH 2020	Sin tokens	12.77	121.79	2920.11	49.59	0.0000	alto recall, muy baja precisión
CSIC/TorpEda 2012	Base	3.9211	124.56	436.51	9.95	0.0000	baseline claramente superior
CSIC/TorpEda 2012	Sin tokens	4.8011	122.88	437.49	9.53	0.0000	predicción excesivamente expansiva

La lectura del bloque multietiqueta requiere más cautela que la de los otros dos. En CSIC 2010, el propio artefacto declara una tarea efectiva `binary_reduced`; en otras palabras, el pipeline multietiqueta se conserva por consistencia metodológica, pero el problema observado en prueba ya no es genuinamente multietiqueta. En ese contexto, el modelo base gana con claridad en F1 micro, Jaccard micro y exact match, aunque la ablación obtenga mayor recall positivo y mejor balanced accuracy en la lectura binaria derivada. La diferencia entre ambas lecturas no es una contradicción: una cosa es maximizar coincidencia global bajo predicción más conservadora y otra maximizar recuperación de la clase positiva aceptando muchos más falsos positivos.

En SR-BH 2020 aparece el caso más exigente del estudio. Allí la variante base domina ampliamente en F1 micro, exact match, Jaccard y precisión micro, mientras que la ablación solo conserva como aparente ventaja un recall micro muy alto acompañado por una caída drástica de precisión. El patrón es claro: sin tokens sospechosos, el modelo pasa a sobrepredecir etiquetas con una agresividad que destruye la coincidencia exacta y el solapamiento útil entre conjuntos reales y predichos.

TorpEda 2012 muestra un comportamiento similar. Aunque la tarea se declara como multietiqueta, el mismo artefacto advierte que la cardinalidad real observada en prueba equivale en la práctica a una sola etiqueta o instancia vacía. Aun así, la comparación sigue siendo informativa: el baseline mantiene la mejor combinación entre F1 micro, Jaccard, exact match y AUC, mientras que la ablación incrementa fuertemente la cardinalidad predicha y la proporción de instancias clasificadas como multietiqueta, produciendo una salida excesivamente expansiva.

8.6. Síntesis transversal

La comparación completa confirma que ninguna formulación domina de manera universal a las otras. El bloque one-class cumple mejor cuando la pregunta principal es si una solicitud se aparta o no del soporte de normalidad. El bloque multiclase es el más eficiente para tipificar cuando las clases están suficientemente definidas y la inferencia debe permanecer liviana. El bloque multietiqueta conserva su mayor valor analítico en SR-BH 2020, donde la riqueza semántica del problema justifica el costo adicional y donde la simplificación a una sola etiqueta empobrecería la lectura del fenómeno.

También se observa que la utilidad de las variables de tokens sospechosos depende del escenario. En Harvard multiclase y multietiqueta, y en TorpEda multiclase y multietiqueta, esas variables aportan una fracción visible del rendimiento. En cambio, en CSIC multiclase y en parte del bloque one-class la ablación genera cambios menores o incluso mejoras marginales. Por eso no corresponde defender una postura absoluta; lo defendible es mostrar la evidencia de ambas variantes y discutir cuándo esas señales funcionan como atajos demasiado específicos y cuándo constituyen información realmente útil.

Por último, el costo operativo vuelve a separar tareas. Multiclase es, en general, la formulación más liviana durante la aplicación. One-class conserva tiempos de entrenamiento muy bajos, pero con una huella de memoria mucho más alta. Multietiqueta es la formulación más costosa en SR-BH 2020, aunque también la más fiel a la estructura semántica del dataset. El resultado global del capítulo, por tanto, no es la proclamación de un único mejor modelo, sino la delimitación bastante precisa de qué enfoque conviene según el tipo de etiqueta disponible, el costo operativo tolerable y el grado de riqueza semántica que se desea preservar.

8.7. Limitaciones de las corridas

La primera limitación relevante es de soporte de clases. En Harvard multiclase, la clase CAPEC-248 Command Injection no queda representada en el conjunto de prueba y, además, el dataset completo solo aporta un registro de esa categoría. En términos prácticos, esto impide evaluar esa clase con estabilidad estadística y obliga a leer con cautela cualquier promedio macro que dependa de etiquetas declaradas pero ausentes en test. El problema no invalida la corrida, pero sí limita la interpretación de la cobertura multiclase sobre el espacio completo de CAPEC.

La segunda limitación es de granularidad efectiva. CSIC 2010 en configuración multiclase termina comportándose como un problema de dos etiquetas efectivas en prueba; CSIC 2010 y TorpEda 2012 en configuración multietiqueta se reducen de hecho a escenarios casi monolabel. Estas corridas siguen siendo útiles para sostener la matriz experimental completa y para contrastar el comportamiento del pipeline bajo tareas formalmente distintas, pero no deben presentarse como evidencia de multietiqueta genuina en el mismo sentido que SR-BH 2020.

La tercera limitación es el alcance del tuning. Todas las corridas utilizaron presupuestos pequeños. Eso es coherente con el carácter reproducible y acotado del estudio, pero no autoriza a afirmar que se halló el óptimo global de cada familia de modelos. Los resultados son comparables dentro del protocolo definido; no constituyen, sin embargo, una búsqueda exhaustiva del máximo desempeño posible.

La cuarta limitación es de entorno operativo. Los benchmarks informan latencia, throughput, CPU, memoria, tamaño del modelo y tasa de fallos sobre un pipeline local de

extracción de características más inferencia. Esa evidencia es útil para estimar factibilidad, pero no sustituye un despliegue real sobre un proxy reverso, con concurrencia sostenida, tráfico de red, reglas de decisión y costos de integración propios de un WAF completo.

Capítulo 9

Protocolo de evaluación de viabilidad en tiempo real en un entorno tipo WAF

9.1. Requisitos operativos

El alcance no incluye el despliegue de un WAF completo, pero sí exige justificar si los modelos estudiados podrían integrarse, al menos teóricamente, dentro de un entorno de inspección HTTP con restricciones de latencia y de recursos. Desde esa perspectiva, los requisitos mínimos son cuatro: (i) mantener una latencia baja y estable en el pipeline de extracción e inferencia; (ii) sostener un *throughput* razonable bajo carga liviana; (iii) evitar picos de memoria incompatibles con un despliegue práctico; y (iv) conservar artefactos de tamaño acotado, de manera que el costo de persistencia y carga del modelo no domine el sistema.

9.2. Consolidado operativo por dataset y modelo

Cuadro 9.1: Resumen consolidado de viabilidad operativa.

Tarea	Dataset	Variante	t_{opt} (s)	t_{build} (s)	$t_{apply}^{\dagger}$ (s)	Thr. (req/s)	P95 (ms)	CPU (%)	Peak RSS (MB)	Modelo (KiB)
One-class	CSIC 2010	Base	4.120	1.071	2.076	192.708	9.916	130.56	873.45	569.5
One-class	CSIC 2010	Sin tokens	4.264	1.161	2.429	164.639	10.697	144.47	861.43	561.2
One-class	SR-BH 2020	Base	3.859	5.270	1.629	245.480	6.032	123.97	5448.73	569.5
One-class	SR-BH 2020	Sin tokens	5.142	6.797	1.648	242.650	5.458	124.36	5429.17	561.2
One-class	CSIC/TorpEda 2012	Base	1.330	0.380	1.611	248.260	5.135	146.47	942.27	569.5
One-class	CSIC/TorpEda 2012	Sin tokens	1.238	0.249	1.469	272.250	4.518	159.26	902.93	561.2
Multiclase	CSIC 2010	Base	2.4525	1.0872	1.5929	376.6683	3.2893	114.26	479.48	3.42
Multiclase	CSIC 2010	Sin tokens	2.3335	1.0055	1.4508	413.5759	2.6209	118.56	467.23	3.08
Multiclase	SR-BH 2020	Base	10.0885	137.3066	1.5420	388.8543	2.9597	119.90	2380.93	6.87
Multiclase	SR-BH 2020	Sin tokens	9.5461	144.5713	1.5916	376.9847	3.1997	118.12	2269.09	6.12
Multiclase	CSIC/TorpEda 2012	Base	3.8222	14.6148	1.4965	400.9321	2.8933	118.28	478.72	5.51
Multiclase	CSIC/TorpEda 2012	Sin tokens	3.7528	12.0153	1.5008	399.7902	2.8655	116.61	472.93	4.88
Multietiqueta	CSIC 2010	Base	0.3287	2.4590	2.0293	295.67	4.2743	132.07	445.46	4.17
Multietiqueta	CSIC 2010	Sin tokens	0.3287	1.9575	1.9460	308.33	4.2951	122.30	445.56	3.85
Multietiqueta	SR-BH 2020	Base	2.98	19.96	6.57	91.32	13.60	122.52	2862.48	50.76
Multietiqueta	SR-BH 2020	Sin tokens	N/A	37.69	6.36	94.29	12.77	121.79	2920.11	49.59
Multietiqueta	CSIC/TorpEda 2012	Base	0.7330	2.6578	2.2321	268.84	3.9211	124.56	436.51	9.95
Multietiqueta	CSIC/TorpEda 2012	Sin tokens	0.7330	1.5397	2.3763	252.49	4.8011	122.88	437.49	9.53

Aclaración de lectura. En la fila *Multietiqueta / SR-BH 2020 / Sin tokens*, la celda N/A en t_{opt} indica que la corrida fue ejecutada sin búsqueda de hiperparámetros. No se trata de un tiempo de optimización igual a cero, sino de una etapa omitida por diseño. La implementación de la ablación multietiqueta ejecuta la variante sin tokens sospechosos con el ajuste de hiperparámetros desactivado salvo que se solicite explícitamente una reoptimización. En esa configuración, las diferencias observadas entre la variante base y la variante sin tokens quedan asociadas a la remoción de variables léxicas sospechosas y no a una segunda búsqueda que pudiera modificar el resultado por otra vía. Por esa razón, el valor correcto es *no aplica*. Mantener esta distinción impide leer como eficiencia un bloque de *tuning* que no fue ejecutado.

9.3. Lectura de factibilidad

El consolidado operativo permite distinguir con bastante claridad tres perfiles. El primero es el de las corridas multiclase sobre CSIC y TorpEda: alta velocidad, latencias muy bajas, artefactos pequeños y memoria moderada. Son las variantes más cómodas para una integración tipo WAF si la tarea de interés es tipificar una única clase por solicitud. El segundo perfil es el de las corridas one-class: su entrenamiento final es rápido y su discusión metodológica es atractiva cuando se quiere entrenar solo con tráfico normal, pero la memoria de proceso es considerablemente mayor, especialmente en Harvard. El tercer perfil es el de las corridas multietiqueta sobre SR-BH 2020: son las más costosas, pero también las que preservan mejor la estructura semántica del dataset más realista del estudio.

Desde una lectura estrictamente operacional, el cuello de botella más severo del conjunto no está en la inferencia multiclase, sino en la memoria de one-class y en la latencia/throughput de multietiqueta sobre Harvard. Aun así, ninguna corrida muestra tasa de fallos distinta de cero en los benchmarks reportados, lo que permite sostener que, al menos dentro del alcance local del protocolo evaluado, no aparecen señales de inestabilidad catastrófica del pipeline.

La consecuencia práctica es que la factibilidad teórica sí puede defenderse, pero de forma condicionada. Si el objetivo fuera un componente liviano y de baja latencia para clasificación de ataques con etiqueta única, las variantes multiclase resultan las candidatas más naturales. Si el objetivo fuera una alarma de anomalía entrenada solo con normales, el esquema one-class sigue siendo defendible, aunque con un costo de memoria que debe declararse. Si lo que se busca es conservar la semántica CAPEC multietiqueta de SR-BH 2020, entonces la formulación multietiqueta es la más fiel al problema, pero no la más barata de desplegar.

Capítulo 10

Conclusiones y trabajo futuro

10.1. Conclusiones

10.2. Trabajos futuros

Apéndice A

Comandos reproducibles

Bibliografía

- [1] OWASP Community. *Web Application Firewall*. https://owasp.org/www-community/Web_Application_Firewall.
- [2] ModSecurity Project. *Open Source Web Application Firewall*. <https://modsecurity.org>.
- [3] MITRE. *CAPEC (Common Attack Pattern Enumeration and Classification)*. <https://capec.mitre.org>.
- [4] MITRE. *About CAPEC*. <https://capec.mitre.org/about/index.html>.
- [5] MITRE. *CAPEC Schema (XML) documentation*. <https://capec.mitre.org/documents/schema/index.html>.
- [6] MITRE. *CAPEC Schema Description (v2.5)*. https://capec.mitre.org/documents/documentation/CAPEC_Schema_Description_v2.5.pdf.
- [7] OWASP Community. *OWASP Risk Rating Methodology*. https://owasp.org/www-community/OWASP_Risk_Rating_Methodology.
- [8] Sureda Riera, T. et al. *SR-BH 2020 multi-label dataset*. Harvard Dataverse. DOI: <https://doi.org/10.7910/DVN/OGOIXX>.
- [9] Sureda Riera, T. et al. *A new multi-label dataset for Web attacks CAPEC classification using machine learning techniques*. Expert Systems with Applications, 2022.
- [10] Kruegel, C., Vigna, G. *Anomaly Detection of Web-based Attacks*. ACM CCS, 2003. https://sites.cs.ucsb.edu/~vigna/publications/2003_kruegel_vigna_ccs03.pdf.
- [11] Rifkin, R., Klautau, A. *In Defense of One-Vs-All Classification*. JMLR, 2004. <https://www.jmlr.org/papers/volume5/rifkin04a/rifkin04a.pdf>.
- [12] Tsoumakas, G., Katakis, I. *Multi-Label Classification: An Overview*. 2007.
- [13] James, G., Witten, D., Hastie, T., Tibshirani, R. *An Introduction to Statistical Learning*. Cap. 4 (Logistic Regression).
- [14] scikit-learn developers. *LogisticRegression documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.
- [15] scikit-learn developers. *GridSearchCV documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html.

- [16] scikit-learn developers. *RandomizedSearchCV documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html.
- [17] scikit-learn developers. *Permutation importance documentation*. https://scikit-learn.org/stable/modules/permutation_importance.html.
- [18] scikit-learn developers. *SGDOneClassSVM documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDOneClassSVM.html.
- [19] scikit-learn developers. *Kernel Approximation documentation (Nystroem / RBFSampler)*. https://scikit-learn.org/stable/modules/kernel_approximation.html.
- [20] scikit-learn developers. *Successive Halving Iterations documentation*. https://scikit-learn.org/stable/modules/grid_search.html#searching-for-optimal-parameters-with-successive-halving.
- [21] IMPACT Cyber Trust. *HTTP DATASET CSIC 2010*. https://www.impactcybertrust.org/dataset_view?idDataset=940.
- [22] Torrano-Giménez, C., Pérez, A., Álvarez, G. *TORPEDA: Una especificación abierta de conjuntos de datos de ataques a aplicaciones web*. RECSI 2012. https://recsi2012.mondragon.edu/es/programa/recsi2012_submission_73.pdf.
- [23] Information Security Institute (CSIC). *HTTP DATASET CSIC 2010 (dataset description)*. https://petescully.co.uk/wp-content/uploads/2018/04/http_dataset_csic_2010.pdf.
- [24] Epp, N., Funk, R., Cappel, C. *Anomaly-based Web Application Firewall using HTTP-specific features and One-Class SVM* (artefacto/código, Zenodo). <https://zenodo.org/records/1336812>, 2018.
- [25] Riverol, A., Betarte, G., Martínez, R., Pardo, Á. *Capturing the security expert knowledge in feature selection for web application attack detection*. LADC 2024. arXiv:2407.18445.
- [26] Fang, M., Wang, Y., Yang, L., Wu, H., Yin, Z., Liu, X., Xie, Z., Kong, Z. *Reinventing web security: An enhanced cycle-consistent generative adversarial network approach to intrusion detection*. Electronics, 13(9):1711, 2024.
- [27] Zhou, Y., Zhu, H., Chai, Y., Jiang, Y., Liu, Y. *Towards Trustworthy Web Attack Detection: An Uncertainty-Aware Ensemble Deep Kernel Learning Model*. arXiv:2410.07725, 2024.
- [28] Saito, T., Rehmsmeier, M. *The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets*. PLOS ONE 10(3):e0118432, 2015. DOI: <https://doi.org/10.1371/journal.pone.0118432>.
- [29] Chicco, D., Jurman, G. *The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation*. BMC Genomics 21, 6 (2020). DOI: <https://doi.org/10.1186/s12864-019-6413-7>.

- [30] Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., Williamson, R. C. *Estimating the Support of a High-Dimensional Distribution*. Neural Computation, 13(7), 2001.
- [31] Williams, C. K. I., Seeger, M. *Using the Nyström Method to Speed Up Kernel Machines*. Advances in Neural Information Processing Systems 13, 2001.
- [32] Bergstra, J., Bengio, Y. *Random Search for Hyper-Parameter Optimization*. Journal of Machine Learning Research, 13, 2012.
- [33] Brodersen, K. H., Ong, C. S., Stephan, K. E., Buhmann, J. M. *The Balanced Accuracy and Its Posterior Distribution*. Proceedings of the 20th International Conference on Pattern Recognition, 2010.